

DATABASE PHASE 2

SUPERVISION

- DR. NIVIN ATEF - PROFESSOR
- ENG. AMIRA FEKRY - TEACHING ASSISTANT

TEAM MEMBERS

- ADHAM HISHAM KANDIL (22P0217)
- YOUSIF SALAH MOHAMMED (22P0232)
- SAIFELDIN TAMER ELSAYES (22P0233)
- AHMED MOHAMED TALAAT (22P0176)
- JANA SAMEH SAMIR (22P0237)
- OMAR HASSAN ELSAYED (22P0167)

TABLE OF CONTENTS

1. Vehicle Rental System - Database Project Report	4
1.1. Introduction	4
1.2. Entity Relationship Diagram (ERD)	4
1.3. Database Schema Diagram	5
1.4. SQL Implementation Details	5
1.3.1. Table Creation and Constraints	5
1.3.2. Indexes for Performance Optimization	10
1.3.3. Sample Data Insertion	10
1.3.4. Example Queries	11
1.4. Summary of Database Implementation	12
2. User Interface Implementation	12
2.1. Common Database Connection Pattern	12
2.2. Customer Control	13
2.3. Employee Control	15
2.4. Garage Control	18
2.5. Maintenance Control	23
2.6. Payment Control	25
2.7. Rentals Control	26
2.8. Reviews Control	28
2.9. Vehicle Control	30
2.10. VehicleMaintenance Control	33
2.11. VehicleRental Control	35
2.12. Dashboard Overview	37
2.13. Summary of User Interface Implementation	38
3. Error Handling and Data Validation	38
3.1. Database Connection Error Handling	38
3.2. Exception Handling for Database Operations	39
3.3. Data Type Validation	40
3.4. Input Format Validation	41
3.5. Required Field Validation	42
3.6. Range and Value Validation	43
3.7. UI State Validation	44
3.8. User Feedback	45
4. Stored Procedures Implementation	47
4.1. Insert Procedures	47
4.2. Update Procedures	48
4.3. Delete Procedures	49
4.4. Select/Get Procedures	49
4.5. Procedure Organization	50

5. Stored Procedure Testing.....	50
5.1. Test Structure and Organization.....	50
5.2. Customer Procedure Tests	50
5.3. Payment Procedure Tests	51
5.4. Maintenance Procedure Tests	52
5.5. Review Procedure Tests	53
5.6. Rental Procedure Tests.....	53
5.7. Test Execution and Verification	54
6. Dashboard - Centralized User Interface.....	54
6.1. Dashboard Implementation.....	54
6.2. Integrated Controls.....	55
6.3. Centralized Architecture Benefits	56

1. VEHICLE RENTAL SYSTEM - DATABASE PROJECT REPORT

1.1. INTRODUCTION

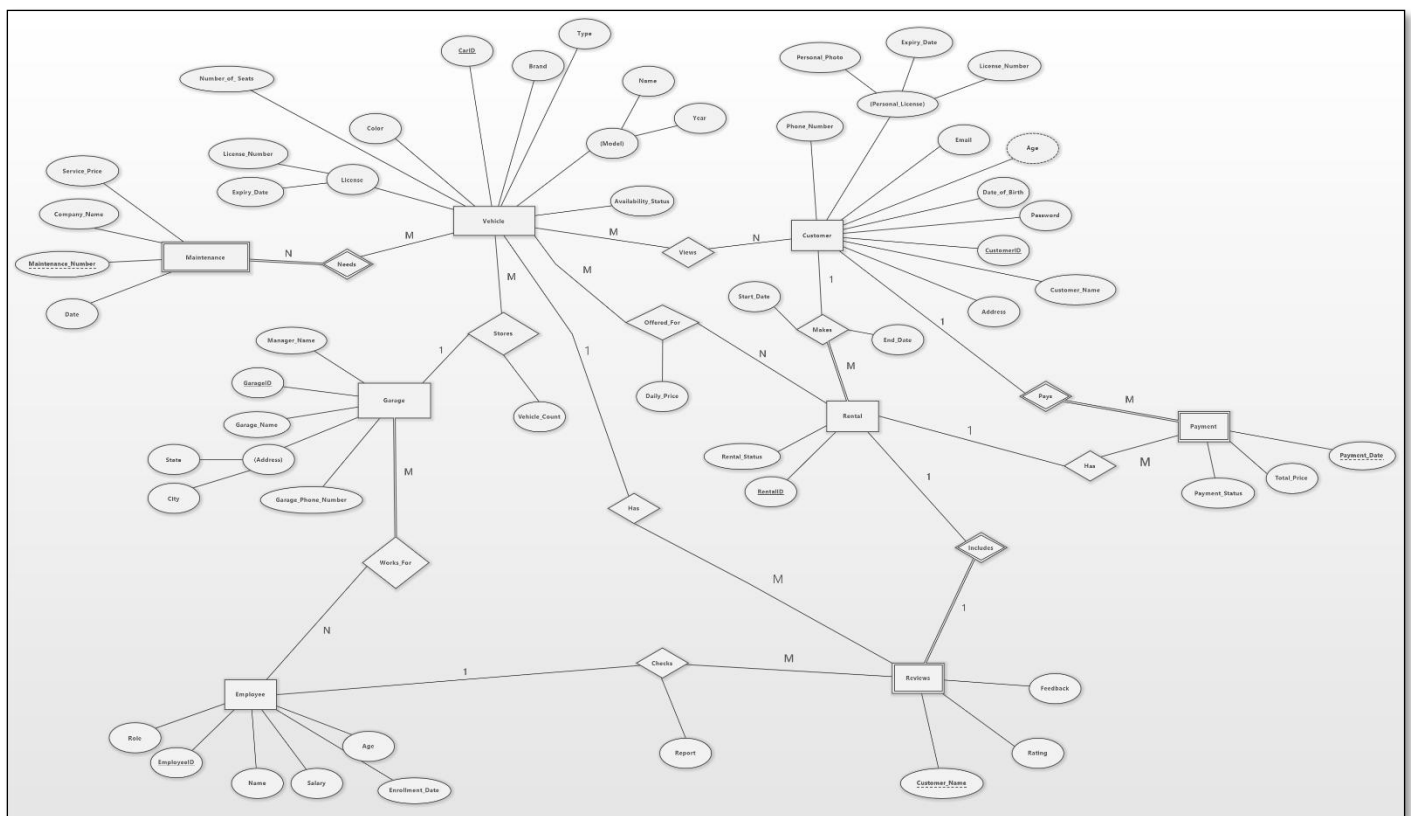
The Vehicle Rental System is a comprehensive database management system designed to efficiently handle the operations of a vehicle rental business. This system manages various aspects of the rental process, from customer registration to vehicle maintenance, ensuring smooth operation of the business.

The system is built around several key entities that form the backbone of the database architecture:

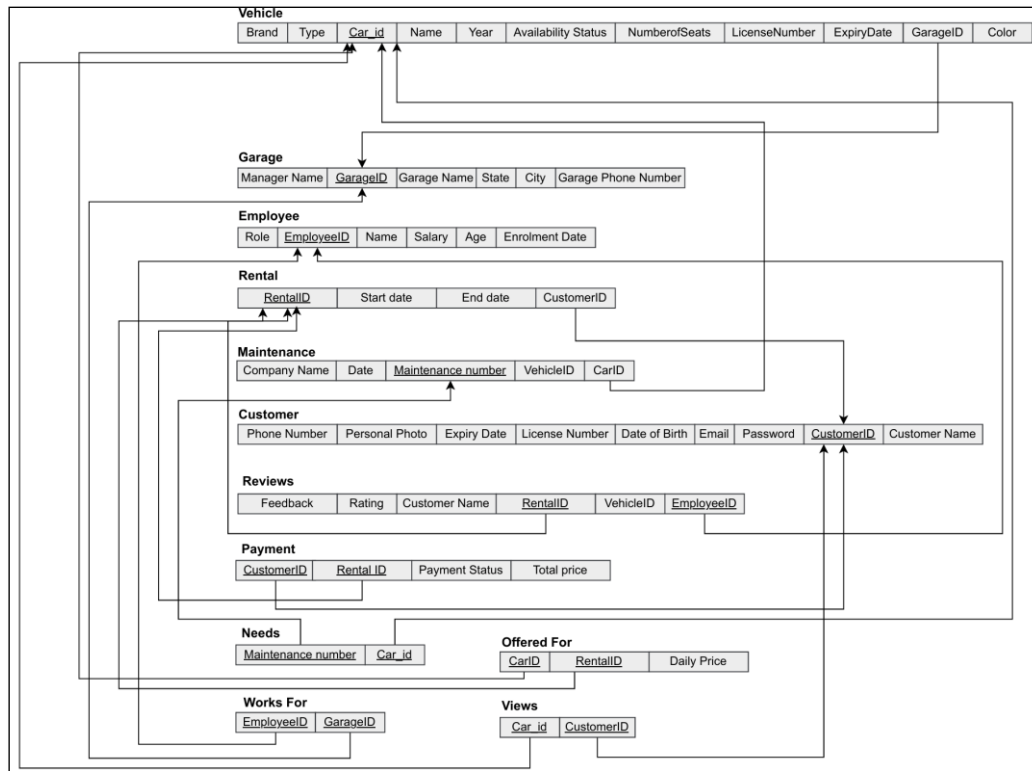
1. **Customer** - Individuals who rent vehicles, including their personal and license information
2. **Vehicle** - Details about the available cars for rent, including model, make, and availability status
3. **Garage** - Storage facilities where vehicles are kept when not rented
4. **Employee** - Staff members who manage various aspects of the rental business
5. **Rental** - Records of vehicle rental transactions
6. **Payment** - Financial transactions associated with rentals
7. **Maintenance** - Service records for vehicles
8. **Review** - Customer feedback about rental experiences

The system implements a relational database model with appropriate constraints and relationships between entities to maintain data integrity. Stored procedures are used to encapsulate complex business logic and ensure consistent data manipulation across the application.

1.2. ENTITY RELATIONSHIP DIAGRAM (ERD)



1.3. DATABASE SCHEMA DIAGRAM



This diagram presents the complete database schema including tables, columns, data types, constraints, and relationships between entities. It provides a comprehensive visual representation of how the data is structured within the Vehicle Rental System database.

1.4. SQL IMPLEMENTATION DETAILS

This section explains the SQL code implementation of the Vehicle Rental System database, including table creation, constraints, indexes, and sample data.

1.3.1. TABLE CREATION AND CONSTRAINTS

1.3.1.1. CUSTOMER TABLE

```
CREATE TABLE Customer
(
    CustomerID          INT IDENTITY (1,1) PRIMARY KEY,
    Customer_Name       VARCHAR(100) NOT NULL,
    Address             VARCHAR(255),
    Date_of_Birth       DATE CHECK (Date_of_Birth <= GETDATE()),
    Age                 INT CHECK (Age >= 18 AND Age <= 100),
    Email               VARCHAR(100) NOT NULL,
    Phone_Number        VARCHAR(20),
    Password            VARCHAR(100) NOT NULL,
    License_Number       VARCHAR(50) NOT NULL,
    License_Expiry_Date DATE NOT NULL CHECK (License_Expiry_Date > GETDATE(
)),
    License_Personal_Photo VARCHAR(MAX),
    CONSTRAINT CHK_Email_Valid CHECK (Email LIKE '%@gmail.com' OR Email LIKE '%@yahoo.
com')
);
```

Explanation:

- Creates the Customer table with an auto-incrementing primary key
- Implements check constraints to ensure:
 - Date of birth is not in the future
 - Customer age is between 18 and 100
 - License has not expired
 - Email format is valid (must be from gmail.com or yahoo.com)
- Requires essential fields like name, email, password, and license information

1.3.1.2. RENTAL TABLE**CREATE TABLE Rental**

```
(
    RentalID      INT IDENTITY (1,1) PRIMARY KEY,
    Start_Date    DATE NOT NULL,
    End_Date      DATE NOT NULL,
    Rental_Status VARCHAR(50) CHECK (Rental_Status IN ('Pending', 'Confirmed', 'Cancelled', 'Completed')),
    CustomerID    INT NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customer (CustomerID),
    CONSTRAINT CHK_Rental_Period CHECK (End_Date > Start_Date)
);
```

Explanation:

- Creates the Rental table to track rental transactions
- Ensures rental end date is after start date
- Limits rental status to predefined values
- Links each rental to a specific customer through a foreign key

PAYMENT TABLE**CREATE TABLE Payment**

```
(
    PaymentID      INT IDENTITY (1,1) PRIMARY KEY,
    Total_Price     DECIMAL(10, 2) NOT NULL CHECK (Total_Price >= 0),
    Payment_Status  VARCHAR(50) CHECK (Payment_Status IN ('Pending', 'Paid', 'Failed'))
,
    Payment_Date    DATE NOT NULL,
    CustomerID      INT NOT NULL,
    RentalID        INT NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customer (CustomerID),
    FOREIGN KEY (RentalID) REFERENCES Rental (RentalID)
);
```

Explanation:

- Tracks financial transactions for rentals
- Ensures price is non-negative
- Restricts payment status to valid values
- Links payments to both customers and rental records

MAINTENANCE TABLE

CREATE TABLE Maintenance

```
(
    Maintenance_Number INT IDENTITY (1,1) PRIMARY KEY,
    Service_Price       DECIMAL(10, 2) NOT NULL CHECK (Service_Price >= 0),
    Company_Name        VARCHAR(100)   NOT NULL,
    Date                DATE           NOT NULL CHECK (Date <= GETDATE())
);
```

Explanation:

- Records vehicle maintenance services
- Ensures service price is non-negative
- Requires maintenance date to be in the past or present

GARAGE TABLE

CREATE TABLE Garage

```
(
    GarageID            INT IDENTITY (1,1) PRIMARY KEY,
    Garage_Name          VARCHAR(100) NOT NULL,
    Address              VARCHAR(255) NOT NULL,
    City                 VARCHAR(100) NOT NULL,
    State                VARCHAR(100) NOT NULL,
    Manager_Name         VARCHAR(100) NOT NULL,
    Garage_Phone_Number  VARCHAR(20)  NOT NULL,
    Vehicle_Count        INT DEFAULT 0 CHECK (Vehicle_Count >= 0),
    CONSTRAINT CHK_Phone_Format CHECK (
        Garage_Phone_Number LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]' OR
        Garage_Phone_Number LIKE '[0-9][0-9][0-9]-[0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]'
    );
```

Explanation:

- Stores information about vehicle storage locations
- Sets a default vehicle count of zero
- Implements a check constraint to ensure phone numbers follow specific formats
- Requires essential information like address and manager name

VEHICLE TABLE

CREATE TABLE Vehicle

```
(
    CarID               INT IDENTITY (1,1) PRIMARY KEY,
    Brand                VARCHAR(50)  NOT NULL,
    Type                 VARCHAR(50)  NOT NULL,
    Model_Name           VARCHAR(100) NOT NULL,
    Model_Year           INT           NOT NULL,
    Color                VARCHAR(30)  NOT NULL,
    Availability_Status  VARCHAR(10)  NOT NULL,
    Number_of_Seats      INT           NOT NULL,
    License_Number       VARCHAR(6)   NOT NULL,
);
```

```

License_Expiry_Date DATE NOT NULL,
GarageID INT NOT NULL,
CONSTRAINT CHK_Vehicle_CarID_Positive CHECK (CarID > 0),
CONSTRAINT CHK_Vehicle_ModelYear_Valid CHECK (Model_Year BETWEEN 1950 AND YEAR(GET
DATE()))),
CONSTRAINT CHK_Vehicle_LicenseNumber_Format CHECK (
    License_Number LIKE '[A-Z][A-Z][A-Z][0-9][0-9][0-9]' AND
    LEN(License_Number) = 6
),
CONSTRAINT CHK_Vehicle_Status_Valid CHECK (
    Availability_Status IN ('yes', 'no', 'Yes', 'No', 'YES', 'NO')
),
CONSTRAINT FK_Vehicle_Garage FOREIGN KEY (GarageID) REFERENCES Garage (GarageID)
);

```

Explanation:

- Stores detailed information about rental vehicles
- Enforces license plate format (3 uppercase letters followed by 3 digits)
- Restricts model year to be between 1950 and the current year
- Limits availability status to yes/no values (case insensitive)
- Associates each vehicle with a garage location

JUNCTION TABLES

Vehicle_Needs_Maintenance:

```

CREATE TABLE Vehicle_Needs_Maintenance
(
    Maintenance_Number INT,
    CarID INT,
    PRIMARY KEY (Maintenance_Number, CarID),
    FOREIGN KEY (Maintenance_Number) REFERENCES Maintenance (Maintenance_Number),
    FOREIGN KEY (CarID) REFERENCES Vehicle (CarID)
);

```

Explanation:

- Implements a many-to-many relationship between vehicles and maintenance records
- Uses a composite primary key of both maintenance number and car ID

Vehicle_Offered_For_Rental:

```

CREATE TABLE Vehicle_Offered_For_Rental
(
    RentalID INT NOT NULL,
    CarID INT NOT NULL,
    Daily_Price DECIMAL(10, 2) NOT NULL,
    CONSTRAINT PK_VehicleRental PRIMARY KEY (RentalID, CarID),
    CONSTRAINT CHK_VehicleRental_DailyPrice_Positive CHECK (Daily_Price >= 0),
    CONSTRAINT FK_VehicleRental_Rental FOREIGN KEY (RentalID) REFERENCES Rental (RentalID),
    CONSTRAINT FK_VehicleRental_Car FOREIGN KEY (CarID) REFERENCES Vehicle (CarID)
);

```


Explanation:

- Implements a many-to-many relationship between rentals and vehicles
- Includes the daily price for each vehicle in a rental
- Ensures the daily price is non-negative

Employee_Works_For_Garage:

```
CREATE TABLE Employee_Works_For_Garage
(
    EmployeeID INT,
    GarageID INT,
    PRIMARY KEY (EmployeeID, GarageID),
    FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) ON DELETE CASCADE,
    FOREIGN KEY (GarageID) REFERENCES Garage (GarageID) ON DELETE CASCADE
);
```

Explanation:

- Implements a many-to-many relationship between employees and garages
- Uses CASCADE delete to automatically remove employee-garage assignments when either an employee or garage is deleted

EMPLOYEE AND REVIEWS TABLES**Employee Table:**

```
CREATE TABLE Employee
(
    EmployeeID INT IDENTITY (1,1) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Age INT CHECK (Age >= 18 AND Age <= 65),
    Role VARCHAR(50),
    Salary DECIMAL(10, 2) CHECK (Salary >= 0),
    Enrollment_Date DATE NOT NULL
);
```

Explanation:

- Stores information about staff members
- Ensures employees are of legal working age (18-65)
- Guarantees salary is non-negative

Reviews Table:

```
CREATE TABLE Reviews
(
    ReviewID INT IDENTITY (1,1) PRIMARY KEY,
    RentalID INT NOT NULL,
    Customer_Name VARCHAR(100),
    Feedback VARCHAR(MAX),
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    Checked_By INT,
    Check_Report VARCHAR(MAX),
    FOREIGN KEY (RentalID) REFERENCES Rental (RentalID),
```

```
FOREIGN KEY (Checked_By) REFERENCES Employee (EmployeeID)
);
```

Explanation:

- Stores customer feedback about rental experiences
- Limits ratings to a 1-5 scale
- Links reviews to specific rentals
- Tracks which employee verified the review

1.3.2. INDEXES FOR PERFORMANCE OPTIMIZATION

The database uses various indexes to improve query performance:

```
CREATE INDEX IDX_Customer_Name ON Customer (Customer_Name);
CREATE INDEX IDX_Customer_Email ON Customer (Email);
CREATE INDEX IDX_Rental_CustomerID ON Rental (CustomerID);
CREATE INDEX IDX_Payment_CustomerID ON Payment (CustomerID);
CREATE INDEX IDX_Payment_RentalID ON Payment (RentalID);
CREATE INDEX IDX_Maintenance_Company_Name ON Maintenance (Company_Name);
CREATE INDEX IDX_Garage_Name ON Garage (Garage_Name);
CREATE INDEX IDX_VNM_Maintenance_Number ON Vehicle_Needs_Maintenance (Maintenance_Number);
CREATE INDEX IDX_VNM_CarID ON Vehicle_Needs_Maintenance (CarID);
CREATE INDEX IDX_Employee_Name ON Employee (Name);
CREATE INDEX IDX_Review_RentalID ON Reviews (RentalID);
CREATE INDEX IDX_Review_CheckedBy ON Reviews (Checked_By);
CREATE INDEX IDX_EWG_EmployeeID ON Employee_Works_For_Garage (EmployeeID);
CREATE INDEX IDX_EWG_GarageID ON Employee_Works_For_Garage (GarageID);
```

Explanation:

- These indexes improve search performance for frequently queried fields
- They particularly optimize searches on foreign keys and commonly filtered attributes
- Secondary indexes like those on customer names and emails speed up searches without altering the physical order of the data

1.3.3. SAMPLE DATA INSERTION

The database includes sample data to demonstrate its functionality and for testing purposes:

```
-- CUSTOMER
INSERT INTO Customer (Customer_Name, Address, Date_of_Birth, Age, Email, Phone_Number,
    Password, License_Number,
    License_Expiry_Date, License_Personal_Photo)
VALUES ('Ahmed Ali', '123 Tahrir St, Cairo', '1995-06-15', 29, 'ahmed.ali@gmail.com',
    '01234567890',
    'hashed_password_123', 'L-987654', '2026-12-31', NULL),
    ('Mona Youssef', '456 Zamalek Rd, Giza', '1988-03-10', 36, 'mona88@yahoo.com',
    '01111222333', 'secure_pass_456',
    'L-123456', '2027-07-15', NULL);
```

Explanation:

- Inserts two customers with complete personal information
- Uses null for the optional license photo field

Similar INSERT statements are included for:

- Rental records
- Payment records
- Maintenance records
- Garage information
- Vehicle data
- Vehicle maintenance relationships
- Employee information
- Employee-garage assignments
- Customer reviews

1.3.4. EXAMPLE QUERIES

The database includes several example queries that demonstrate common operations:

--Select all garages with their managers and vehicle counts

```
SELECT Garage_Name, Address, City, State, Manager_Name, Garage_Phone_Number, Vehicle_Count
FROM Garage
ORDER BY Vehicle_Count DESC;
```

Explanation:

- Retrieves a list of all garages with their details
- Orders the results by vehicle count in descending order (highest capacity first)

--Find employees working at a specific garage

```
SELECT e.Name, e.Role, g.Garage_Name
FROM Employee e
      JOIN Employee_Works_For_Garage ewg ON e.EmployeeID = ewg.EmployeeID
      JOIN Garage g ON ewg.GarageID = g.GarageID
WHERE g.Garage_Name = 'Downtown Garage';
```

Explanation:

- Uses joins to find all employees working at a specific garage
- Returns employee names, roles, and the garage name
- Demonstrates how to query many-to-many relationships

--Count employees per garage

```
SELECT g.Garage_Name, COUNT(ewg.EmployeeID) as EmployeeCount
FROM Garage g
      LEFT JOIN Employee_Works_For_Garage ewg ON g.GarageID = ewg.GarageID
GROUP BY g.Garage_Name;
```

Explanation:

- Uses a LEFT JOIN to include garages with no employees
- Groups results by garage name
- Counts the number of employees at each garage
- Demonstrates GROUP BY and aggregate functions

1.4. SUMMARY OF DATABASE IMPLEMENTATION

The SQL implementation of the Vehicle Rental System database includes:

11 Tables: Customer, Rental, Payment, Maintenance, Garage, Vehicle, Vehicle_Needs_Maintenance, Employee, Reviews, Vehicle_Offered_For_Rental, and Employee_Works_For_Garage

2. USER INTERFACE IMPLEMENTATION

The system is implemented as a Windows Forms application using C# and ADO.NET for database connectivity. Each entity in the database has a corresponding user control that provides a graphical interface for CRUD (Create, Read, Update, Delete) operations on that entity. This section explains the implementation of each form control, highlighting key aspects of their functionality.

2.1. COMMON DATABASE CONNECTION PATTERN

All form controls share a similar pattern for establishing database connections, using SQL Server as the backend database:

```
SqlConnection conn = new SqlConnection(@"Server=LAPTOP-JF8UCNBK\MSSQLSERVER01,57870;Initial Catalog=DatabaseProject;Integrated Security=True");
```

Explanation:

- This connection string specifies the target SQL Server instance (**LAPTOP-JF8UCNBK\MSSQLSERVER01,57870**), which includes the server name and port number
- **Initial Catalog=DatabaseProject** specifies the database name to connect to
- **Integrated Security=True** indicates that Windows Authentication is used rather than SQL Server authentication, eliminating the need to store database credentials in the code
- This pattern is used consistently across all forms to ensure connection string changes only need to be made in one place
- Creating a SqlConnection object doesn't actually open the connection yet - the connection is only established when the **Open()** method is called

2.2. CUSTOMER CONTROL

The CustomerControl provides interfaces for managing customer information. It implements standard CRUD operations through stored procedures.

CustomerID	Customer_Name	Address	Date_of_Birth	Age	Email	Phone_Number	Password	License_Number	License_Expiry_De	License_Personal_
1	Ahmed Ali	123 Tahrir St, Cairo	6/15/1995	29	ahmed.ali@gmail...	01234567890	hashed_password...	L-987654	12/31/2026	
2	Mona Yousef	456 Zamelek Rd...	3/10/1988	36	mona88@yahoo...	01111222333	secure_pass_456	L-123456	7/15/2027	

Loading Customer Data:

```
private void LoadTable(string tableName)
{
    try
    {
        using (SqlConnection con = new SqlConnection(conn.ConnectionString))
        {
            string query = $"SELECT * FROM {tableName}";
            SqlDataAdapter adapter = new SqlDataAdapter(query, con);
            DataTable dt = new DataTable();
            adapter.Fill(dt);
            dataGridView1.DataSource = dt;
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading data: " + ex.Message);
    }
}
```

Explanation:

- This method implements a generic table loading function that can load any table by name
- The `using` statement ensures the connection is properly disposed of even if an exception occurs (implementing the dispose pattern)
- `SqlDataAdapter` serves as a bridge between the `DataTable` and the database, automatically handling connection opening and closing
- The `Fill()` method executes the query and populates the `DataTable` with the results
- Setting the `dataGridView1.DataSource` property automatically binds all columns and rows from the `DataTable` to the grid
- The parameterized `tableName` is directly inserted into the query using string interpolation with `"SELECT * FROM {tableName}"` which could potentially be a SQL injection vulnerability if not properly validated
- The try-catch block captures and displays any database-related errors, preventing application crashes

Inserting Customer Data:

```
private void btnInsert_Click(object sender, EventArgs e)
{
    try
    {
        using (SqlConnection con = new SqlConnection(conn.ConnectionString))
        {
            SqlCommand cmd = new SqlCommand("InsertCustomer", con)
            {
                CommandType = CommandType.StoredProcedure
            };

            cmd.Parameters.AddWithValue("@Customer_Name", txtCustomerName.Text);
            cmd.Parameters.AddWithValue("@Address", txtAddress.Text);
            cmd.Parameters.AddWithValue("@Date_of_Birth", dtpBirth.Value);
            cmd.Parameters.AddWithValue("@Age", int.Parse(txtAge.Text));
            cmd.Parameters.AddWithValue("@Email", txtEmail.Text);
            cmd.Parameters.AddWithValue("@Phone_Number", txtPhoneNumber.Text);
            cmd.Parameters.AddWithValue("@Password", txtPassword.Text);
            cmd.Parameters.AddWithValue("@License_Number", txtLicenseNumber.Text);
            cmd.Parameters.AddWithValue("@License_Expiry_Date", dtpExpiry.Value);
            cmd.Parameters.AddWithValue("@License_Personal_Photo", txtLicensePhoto.Text);

            con.Open();
            int newCustomerId = Convert.ToInt32(cmd.ExecuteScalar());

            MessageBox.Show($"Customer inserted successfully. CustomerID: {newCustomerId}");
        }

        LoadTable("Customer");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " + ex.Message);
    }
}
```

Explanation:

- This event handler responds to the Insert button click event
- Setting **CommandType = CommandType.StoredProcedure** tells ADO.NET to execute a stored procedure rather than a direct SQL query
- Parameters are added to the command using **AddWithValue()**, which maps form field values to the corresponding stored procedure parameters
- Each parameter is named to match the expected parameter names in the stored procedure
- The connection is explicitly opened with **con.Open()** just before command execution to minimize connection time
- **cmd.ExecuteScalar()** executes the stored procedure and returns the first column of the first row (which contains the newly created CustomerID)

- **Convert.ToInt32()** converts the result to an integer to be displayed in the success message
- After successful insertion, the table is refreshed using the **LoadTable()** method to show the newly added record
- String interpolation in the success message (**\$"Customer inserted successfully. CustomerID: {newCustomerId}"**) creates a user-friendly confirmation message that includes the new ID
- Notice that validation is not performed in this method, suggesting it might be handled elsewhere or is a potential improvement area

2.3. EMPLOYEE CONTROL

The EmployeeControl manages employee information with comprehensive validation to ensure data integrity before database operations.

The screenshot shows a web application dashboard with a tabbed interface. The 'Employee' tab is selected. The 'Employee Details' form contains the following fields:

- Employee ID: 1
- Name: Youssef Sami
- Role: Mechanic
- Age: 30
- Salary: 5000.00
- Enrollment Date: Wednesday, January (dropdown)

Below the form are three buttons: 'Insert', 'Update', and 'Delete'. Below the buttons is a table with the following data:

	EmployeeID	Name	Role	Age	Salary	Enrollment Date
▶	1	Youssef Sami	Mechanic	30	5000.00	2024-01-10
	2	Sara Mahmoud	Reviewer	40	6000.00	2023-11-01
	3	John Doe	Mechanic	35	50000.00	2023-01-15
	4	Jane Smith	Manager	28	60000.00	2022-06-10
	5	Mike Johnson	Technician	40	45000.00	2023-03-20

Employee Data Validation:

```
// check en el name w el role fehom Letters w spaces bas
if (!Regex.IsMatch(txtName.Text, @"^[a-zA-Z ]+$"))
{
    // validation error, el esm lazem ykon Letters bas
    MessageBox.Show("Name must contain only letters and spaces.", "Validation Error",
    MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}
if (!Regex.IsMatch(txtRole.Text, @"^[a-zA-Z ]+$"))
{
    // validation error, el role lazem ykon Letters bas
    MessageBox.Show("Role must contain only letters and spaces.", "Validation Error",
    MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}
if (!int.TryParse(txtAge.Text, out int age) || age < 0)
{
    // validation error, el age lazem ykon rakam sah
```

```

        MessageBox.Show("Age must be a valid positive number.", "Validation Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }
}

```

Explanation:

- This validation code implements multiple data quality checks before allowing database operations
- Regular expressions (**Regex.IsMatch**) are used to ensure that name and role fields contain only alphabetic characters and spaces:
 - The pattern **^[a-zA-Z]+\$** means:
 - **^** - start of string
 - **[a-zA-Z]** - any lowercase letter, uppercase letter, or space
 - **+** - one or more of the preceding characters
 - **\$** - end of string
- For the age field, two validations are performed in a single statement:
 - **int.TryParse()** confirms the input can be converted to an integer (returns false if not)
 - **age < 0** checks that the parsed age is positive
- The combined check with the OR operator (**|**) means the validation fails if either condition is true
- Each validation failure displays a specific, user-friendly error message with a warning icon
- The **return** statement immediately exits the method if validation fails, preventing further processing
- Comments are provided in a mix of English and transliterated Arabic, suggesting a collaborative international development team

Loading Employee Data:

```

private void LoadEmployees()
{
    try
    {
        // awel 7aga bnms7 el rows el adema
        dataGridView.Rows.Clear();
        using (SqlConnection conn = new SqlConnection(connectionString))
        {
            conn.Open();
            // hena bnst3ml el procedure ely esmo GetAllEmployees
            using (SqlCommand cmd = new SqlCommand("GetAllEmployees", conn))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                using (SqlDataReader reader = cmd.ExecuteReader())
                {
                    while (reader.Read())
                    {
                        // kol row mn el db byzhar fel grid
                        dataGridView.Rows.Add(
                            reader["EmployeeID"],
                            reader["Name"],
                            reader["Role"],
                            reader["Age"],
                            reader["Salary"],
                            Convert.ToDateTime(reader["Enrollment_Date"]).ToString("yy

```

```
yy-MM-dd")
        );
    }
}
}
}
}
catch (Exception ex)
{
    // hena el error handling, law 7asal ay moshkela fel db aw el connection
    MessageBox.Show("Error loading employees: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
```

Explanation:

- This method populates the employee data grid view from the database
- Unlike the CustomerControl which uses a DataAdapter, this approach uses a SqlDataReader for more direct control
- The process follows a clear sequence:
 1. First, `Rows.Clear()` removes any existing rows to prevent duplicate data
 2. Then a connection is established and explicitly opened
 3. The GetAllEmployees stored procedure is executed with no parameters
 4. A SqlDataReader retrieves the result set one row at a time
 5. Each row is processed in the `while (reader.Read())` loop
- The grid is populated manually by adding rows one at a time using the `dataGridView.Rows.Add()` method
- Column values are accessed from the reader using string indexers (`reader["ColumnName"]`)
- Date formatting is applied to ensure a consistent date display with `Convert.ToDateTime().ToString("yyyy-MM-dd")`
- Nested `using` statements ensure proper resource cleanup for each level (connection, command, reader)
- Inline comments in transliterated Arabic explain key steps in the process:
 - “First thing we clear the old rows”
 - “Here we use the procedure named GetAllEmployees”
 - “Each row from the db appears in the grid”
 - “This is error handling, if any problem occurs in the db or connection”

2.4. GARAGE CONTROL

The GarageControl manages garage information and demonstrates more complex relationships, including the assignment of employees to garages.

Dashboard

Customer Rental Payment Maintenance Garage Vehicle Vehicle Offered For Rental Vehicle Needs Maintenance Employee Reviews

Garage Name: Downtown Garage Insert Update Delete Count Assign

Address: 123 Main St

City: Cairo

State: Cairo

Manager Name: Jane Smith

Phone Number: 01012345672

Vehicle Count: 15

Employee ID: 1

ID	Garage Name	Address	City	State	Manager
1	Downtown Garage	123 Main St	Cairo	Cairo	Jane Smith
2	Westside Garage	456 Oak Ave	Giza	Giza	John Doe
*					

Employee ID	Name	Role
1	Youssef Sami	Mechanic
3	John Doe	Mechanic
4	Jane Smith	Manager
*		

Database Connection Setup:

```
private void SetupDatabaseConnection()
{
    try
    {
        con = new SqlConnection(
            @"Server=LAPTOP-JF8UCNBK\MSSQLSERVER01,57870;Initial Catalog=DatabaseProject;Integrated Security=True");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Failed to set up database connection: " + ex.Message);
        con = null;
    }
}
```

Explanation:

- This method initializes the database connection at the class level rather than creating connections locally within methods
- The `con` variable is likely a class-level field that will be reused across multiple methods
- Unlike other controls that create connections on demand, this approach creates a single connection object during setup
- The connection is not opened here - just configured - actual opening happens when needed for specific operations
- A try-catch block captures potential connection initialization errors, such as invalid connection string format
- If an error occurs, the connection is explicitly set to null to indicate connection setup failure
- Setting `con = null` allows other methods to check if the connection was successfully initialized
- Unlike methods that use `using` statements, this approach requires explicit opening and closing of the connection in other methods
- This pattern is suitable when the same connection might be needed multiple times within a short timespan

Loading Assigned Employees:

```
private void LoadAssignedEmployees(int garageId)
{
    if (con == null)
    {
        MessageBox.Show("Database connection is not initialized.");
        return;
    }

    tbl_AssignedEmployees.Clear();

    try
    {
        con.Open();
        SqlCommand cmd = new SqlCommand(
            "SELECT e.EmployeeID, e.Name, e.Role " +
            "FROM Employee e " +
            "JOIN Employee_Works_For_Garage ewg ON e.EmployeeID = ewg.EmployeeID " +
            "WHERE ewg.GarageID = @GarageID", con);
        cmd.Parameters.AddWithValue("@GarageID", garageId);
        SqlDataReader reader = cmd.ExecuteReader();

        bool hasData = false;
        while (reader.Read())
        {
            hasData = true;
            DataRow row = tbl_AssignedEmployees.NewRow();
            row["Employee ID"] = reader["EmployeeID"] != DBNull.Value ? Convert.ToInt32(reader["EmployeeID"]) : 0;
            row["Name"] = reader["Name"] != DBNull.Value ? reader["Name"].ToString() : "";
            row["Role"] = reader["Role"] != DBNull.Value ? reader["Role"].ToString() : "";
            tbl_AssignedEmployees.Rows.Add(row);
        }

        reader.Close();
        // ...
    }
    // ...
}
```

Explanation:

- This method demonstrates handling of many-to-many relationships by retrieving all employees assigned to a specific garage
- The method begins with a defensive check for a null connection object, preventing execution if the connection wasn't properly initialized
- **tbl_AssignedEmployees.Clear()** resets the data table before populating it with fresh data
- The SQL query uses a JOIN operation to retrieve employee data across multiple tables:
 - The Employee table for basic employee information (ID, Name, Role)

- The Employee_Works_For_Garage junction table to establish the many-to-many relationship
- The query parameter `@GarageID` is properly added using `AddWithValue()` to prevent SQL injection
- The `bool hasData` variable tracks whether any records were found, potentially for further processing
- For each database field, null checking is performed with `reader["ColumnName"] != DBNull.Value` to safely handle null values
- Default values (0 for integers, empty string for text) are provided when null values are encountered
- Each DataRow must be created with `tbl_AssignedEmployees.NewRow()` before it can be populated and added to the DataTable
- The reader is explicitly closed with `reader.Close()` rather than using a `using` statement
- This method leaves the connection open, suggesting it's intended to be part of a larger operation sequence

Loading Garages:

```
private void LoadGarages()
{
    if (con == null)
    {
        MessageBox.Show("Database connection is not initialized.");
        return;
    }

    try
    {
        con.Open();
        SqlCommand cmd = new SqlCommand(
            "SELECT * FROM Garage", con);
        cmd.CommandType = CommandType.Text;
        SqlDataReader reader = cmd.ExecuteReader();

        tbl_Garages.Clear();

        tbl_Garages.Columns.Clear();

        //add garages tables columns...

        bool hasData = false;
        while (reader.Read())
        {
            hasData = true;
            DataRow row = tbl_Garages.NewRow();
            row["ID"] = reader["GarageID"] != DBNull.Value ? Convert.ToInt32(reader["
            GarageID"]) : 0;
            row["Garage Name"] = reader["Garage_Name"] != DBNull.Value ? reader["
            Garage_Name"].ToString() : "";
            row["Address"] = reader["Address"] != DBNull.Value ? reader["Address"].
            ToString() : "";

            row["City"] = reader["City"] != DBNull.Value ? reader["City"].ToString()
            : "";
        }
    }
}
```

```

        row["State"] = reader["State"] != DBNull.Value ? reader["State"].ToString
() : "";
        row["Manager"] = reader["Manager"] != DBNull.Value ? reader["Manager"].
ToString() : "";
        row["Phone Number"] = reader["Phone Number"] != DBNull.Value ? reader["
Phone Number"].ToString() : "";
        row["Vehicle Count"] = reader["Vehicle_Count"] != DBNull.Value ? Convert.
ToInt32(reader["Vehicle_Count"]) : 0;

        tbl_Garages.Rows.Add(row);
    }

    reader.Close();
    // ...
}
// ...
}

```

Explanation:

- This method demonstrates retrieving all garage records from the database for display in the application.
- The method begins with a defensive check for a null connection object, preventing execution if the connection wasn't properly initialized.
- `tbl_Garages.Clear()` and `tbl_Garages.Columns.Clear()` reset the data table and its structure before populating it with fresh data, ensuring no residual data from previous loads.
- The SQL query uses a simple `SELECT * statement to retrieve all columns from the Garage table`, providing a straightforward approach to fetch garage details.
- The query does not involve joins, as it directly accesses the Garage table, focusing on single-table data retrieval.
- The `cmd.CommandType = CommandType.Text` explicitly specifies the command type as a text-based SQL query, enhancing clarity and maintainability.
- The bool `hasData` variable tracks whether any records were found, used to inform the user if no garages are present in the database.
- For each database field, null checking is performed with `reader["ColumnName"] != DBNull.Value` to safely handle null values, preventing runtime errors.
- Default values (0 for integers, empty string for text) are provided when null values are encountered, ensuring consistent data representation.
- Each DataRow must be created with `tbl_Garages.NewRow()` before it can be populated and added to the DataTable, maintaining data table integrity.
- The reader is explicitly closed with `reader.Close()` rather than using a using statement, aligning with manual resource management.
- The method binds the populated `tbl_Garages` to `dgvGarages.DataSource`, updating the UI with the retrieved data.
- A message is displayed if no data is found, providing feedback to the user.
- The `finally` block ensures the database connection is closed if it remains open, preventing resource leaks, though it suggests the method is part of a sequence where the connection state might be managed externally.

Assigning Employees to Garages:

```
private void btnAssignEmployee_Click(object sender, EventArgs e)
{
    if (con == null)
    {
        MessageBox.Show("Database connection is not initialized.");
        return;
    }

    if (dgvGarages.SelectedRows.Count > 0 && !string.IsNullOrEmpty(txtEmployeeID.Text)
)
    {
        if (!int.TryParse(txtEmployeeID.Text, out int employeeId))
        {
            MessageBox.Show("Please enter a valid Employee ID.");
            return;
        }

        var garageIdValue = dgvGarages.SelectedRows[0].Cells["ID"].Value;
        if (garageIdValue == null || !int.TryParse(garageIdValue.ToString(), out int g
arageId))
        {
            MessageBox.Show("Invalid Garage ID selected.");
            return;
        }

        try
        {
            con.Open();
            SqlCommand cmd = new SqlCommand(
                "INSERT INTO Employee_Works_For_Garage (EmployeeID, GarageID) VALUES (
@EmployeeID, @GarageID)", con);
            cmd.Parameters.AddWithValue("@EmployeeID", employeeId);
            cmd.Parameters.AddWithValue("@GarageID", garageId);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Employee assigned to garage successfully!");
        }
        // ...
    }
    // ...
}
```

Explanation:

- This method handles the assignment of employees to garages, implementing the many-to-many relationship between these entities
- The method follows a structured validation sequence before executing the database operation:
 1. First, it checks if the database connection is initialized
 2. Then it verifies that a garage is selected and an employee ID is provided
 3. Next, it validates that the employee ID is a valid integer
 4. Finally, it confirms the selected garage row contains a valid garage ID
- Only after all validations pass does it attempt the database operation
- The code directly uses a SQL INSERT statement rather than a stored procedure:
 - **INSERT INTO Employee_Works_For_Garage (EmployeeID, GarageID) VALUES (@EmployeeID, @GarageID)**
- Despite using direct SQL, proper parameterization is applied to prevent SQL injection

- Parameters are added using **AddWithValue()**, matching parameter names to the placeholders in the query
- ExecuteNonQuery()** is used since no result set is expected from an INSERT operation
- Success feedback is provided to the user through a message box
- The **try** block is shown but the catch block is omitted (indicated by **// ...**), suggesting error handling exists but isn't shown
- Unlike many other methods that use a **using** statement for connection management, this approach:
 1. Explicitly opens the connection with **con.Open()**
 2. Presumably closes it in code not shown or in a finally block
- This pattern is essential for implementing many-to-many relationships in relational databases

The GarageControl demonstrates handling of many-to-many relationships, showing how to assign employees to garages and display the assignments in a secondary grid view.

2.5. MAINTENANCE CONTROL

The MaintenanceControl manages vehicle maintenance records and demonstrates form event wiring and field validation.

The screenshot shows a web application titled "Dashboard" with a navigation bar containing tabs: Customer, Rental, Payment, Maintenance (selected), Garage, Vehicle, Vehicle Offered For Rental, Vehicle Needs Maintenance, Employee, and Reviews.

The "Maintenance Details" form includes the following fields:

- Maintenance Number:
- Company Name:
- Service Price:
- Date:

Below the form are three buttons: Insert, Update, and Delete.

Below the buttons is a table displaying maintenance records:

	Maintenance	Service Price	Company Name	Date
▶	1	500.00	AutoFix Cairo	2025-04-15
	2	750.00	FastTrack Mainte...	2025-04-20

Event Wiring:

```
private void WireUpEvents()
{
    btnInsert.Click += (s, e) => InsertMaintenance();
    btnUpdate.Click += (s, e) => UpdateMaintenance();
    btnDelete.Click += (s, e) => DeleteMaintenance();
    dataGridView.SelectionChanged += (s, e) => FillFieldsFromGrid();
}
```

Explanation:

- This method demonstrates modern C# event wiring using lambda expressions for concise event handler registration
- Each button click event is connected to its corresponding method using the `+=` operator to subscribe to the event
- Lambda expressions `((s, e) =>)` provide a compact syntax for event handling, eliminating the need for separate named methods
- The `s` parameter represents the sender (the control that raised the event)
- The `e` parameter represents the event arguments (which aren't used in these handlers)
- The `dataGridView.SelectionChanged` event automatically updates form fields when a user selects a different row
- This pattern centralizes all event registration in one method, making it easier to understand the control's interactive behavior
- Using lambda expressions with method calls creates a clean separation between event handling and business logic

Filling Form Fields from Selected Grid Row:

```
private void FillFieldsFromGrid()
{
    if (dataGridView.SelectedRows.Count == 0) return;
    var row = dataGridView.SelectedRows[0];
    txtMaintenanceNumber.Text = row.Cells[0].Value.ToString();
    txtServicePrice.Text = row.Cells[1].Value.ToString();
    txtCompanyName.Text = row.Cells[2].Value.ToString();
    dtDate.Value = DateTime.Parse(row.Cells[3].Value.ToString());
}
```

Explanation:

- This method implements automatic form field population based on grid selection - a key usability feature
- The guard clause `if (dataGridView.SelectedRows.Count == 0) return;` prevents null reference exceptions by checking if any row is selected
- For efficiency, only the first selected row (`SelectedRows[0]`) is used even if multiple rows are selected
- The method assumes a specific column order, accessing cells by index rather than by name:
 - Cell[0] for Maintenance Number
 - Cell[1] for Service Price
 - Cell[2] for Company Name
 - Cell[3] for Date
- This approach requires consistent column ordering in the grid to function correctly
- For the date field, the string value is parsed into a `DateTime` object to set the `DateTimePicker` control's value
- No exception handling is present, suggesting `DateTime.Parse` could fail if the database contains invalid date strings

- This method creates bidirectional data flow: database → grid → form fields, enabling a fluid edit workflow

The MaintenanceControl demonstrates clean code organization with separate methods for each CRUD operation and event handling.

2.6. PAYMENT CONTROL

The PaymentControl manages payment transactions and demonstrates the use of DataAdapter for filling DataTable objects.

PaymentID	Total_Price	Payment_Status	Payment_Date	CustomerID	RentalID
1	1500.00	Paid	5/9/2025	1	1
2	1200.00	Pending	5/30/2025	2	2

Loading Payment Data:

```
private void LoadPayments()
{
    try
    {
        using (SqlCommand cmd = new SqlCommand("GetAllPayments", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            SqlDataAdapter adapter = new SqlDataAdapter(cmd);
            DataTable dt = new DataTable();
            adapter.Fill(dt);
            dgvPayments.DataSource = dt;
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading data: " + ex.Message);
    }
}
```

Explanation:

- This method loads all payment records into a DataGridView using a stored procedure
- Unlike the CustomerControl that passes a direct SQL query to the adapter, this approach uses a stored procedure
- The **using** statement is applied to the SqlCommand object rather than the connection, suggesting the connection is managed elsewhere

- The connection object (**conn**) appears to be a class-level field that's used across multiple methods
- Setting **CommandType = CommandType.StoredProcedure** tells ADO.NET to execute "GetAllPayments" as a stored procedure
- The SqlDataAdapter serves as a bridge between the command and the DataTable, eliminating the need to manually:
 - Open the connection
 - Create a reader
 - Loop through results
 - Close the connection
- The adapter's **Fill()** method:
 1. Opens the connection
 2. Executes the command
 3. Fills the DataTable with the results
 4. Closes the connection
- The DataTable is then bound to the DataGridView through the **DataSource** property
- No parameters are passed to the stored procedure, indicating it retrieves all payment records without filtering
- This technique demonstrates a particularly efficient pattern for loading tabular data with minimal code

The PaymentControl demonstrates a streamlined approach to data retrieval using stored procedures and SqlDataAdapter, simplifying the code needed to populate data grids.

2.7. RENTALS CONTROL

The RentalsControl manages rental transactions and demonstrates form loading and unimplemented method placeholders.

	RentalID	Start_Date	End_Date	Rental_Status	CustomerID
▶	1	5/10/2025	5/15/2025	Confirmed	1
	2	6/1/2025	6/5/2025	Pending	2
*					

Form Loading:

```
private void RentalsForm_Load(object sender, EventArgs e)
{
    LoadTable("Rental");
}
```

Explanation:

- This event handler is triggered when the Rentals form is first loaded
- The method calls `LoadTable("Rental")`, suggesting it uses a generic table loading method similar to the one in CustomerControl
- This single line of code demonstrates an effective separation of concerns:
 - The event handler specifies *what* to load
 - The LoadTable method manages *how* to load it
- This approach allows for consistent loading behavior across multiple forms while keeping the implementation details encapsulated
- The pattern supports code reuse and maintainability through smaller, more focused methods
- Event handlers with minimal code are easier to understand and maintain than complex, multi-function handlers

Inserting Rental Records:

```
private void insert_btn_Click(object sender, EventArgs e)
{
    try
    {
        using (var con = new SqlConnection(conn.ConnectionString))
        {
            var cmd = new SqlCommand("InsertRental", con)
            {
                CommandType = CommandType.StoredProcedure
            };

            cmd.Parameters.AddWithValue("@StartDate", start_datepicker.Value);
            cmd.Parameters.AddWithValue("@EndDate", end_datepicker.Value);
            cmd.Parameters.AddWithValue("@Status", status_box.SelectedItem?.ToString()
            ?? "Pending");
            cmd.Parameters.AddWithValue("@CustomerID", int.Parse(customer_id_textbox.T
            ext));

            con.Open();
            int newRentalId = Convert.ToInt32(cmd.ExecuteScalar());

            MessageBox.Show($"Rental inserted successfully. RentalID: {newRentalId}");
        }

        LoadTable("Rental");
    }
    catch (Exception ex)
    {
        MessageBox.Show(@"Error: " + ex.Message);
    }
}
```

Explanation:

- This method handles creating new rental records through a button click event
- It uses modern C# features including:
 - Implicitly typed local variables with `var`

- Object initializers with `{ CommandType = CommandType.StoredProcedure }`
- Null-conditional and null-coalescing operators (`?.` and `??`)
- The parameter values are drawn from various UI controls:
 - DateTimePicker controls for start and end dates (`start_datepicker.Value`)
 - ComboBox for status selection (`status_box.SelectedItem`)
 - TextBox for customer ID (`customer_id_textbox.Text`)
- The expression `status_box.SelectedItem?.ToString() ?? "Pending"` provides a failsafe default value:
 1. If no item is selected (`null`), the null-conditional operator (`?.`) short-circuits
 2. The null-coalescing operator (`??`) then returns "Pending" as the default value
- The `int.Parse()` call for CustomerID lacks validation, making it a potential crash point if non-numeric input is provided
- The newly created rental ID is returned from the stored procedure using `ExecuteScalar()`
- After successful insertion, the grid is refreshed by calling `LoadTable("Rental")`
- The `@` symbol before the error message string (`@"Error: "`) is unnecessary for a simple string but would allow for multi-line formatting

The RentalsControl demonstrates the use of date pickers and combo boxes for data input, showing how different control types can be used to simplify data entry.

2.8. REVIEWS CONTROL

The ReviewsControl manages customer feedback and demonstrates the use of stored procedures for all database operations.

	ReviewID	RentalID	Customer_Name	Feedback	Rating	Checked_By	Check_Report
▶	1	1	Ahmed Ali	The car was clea...	5	2	Verified - Clean c...
	2	2	Mona Youssef	Vehicle had mino...	3	2	Verified - Reporte...
*							

Updating Reviews:

```
private void btnUpdate_Click(object sender, EventArgs e)
{
    try
    {
```

```

using (var con = new SqlConnection(conn.ConnectionString))
{
    var cmd = new SqlCommand("UpdateReview", con)
    {
        CommandType = CommandType.StoredProcedure
    };

    cmd.Parameters.AddWithValue("@ReviewID", int.Parse(txtReviewID.Text));
    cmd.Parameters.AddWithValue("@RentalID", int.Parse(txtRentalID.Text));
    cmd.Parameters.AddWithValue("@Customer_Name", txtCustomerName.Text);
    cmd.Parameters.AddWithValue("@Feedback", txtFeedback.Text);
    cmd.Parameters.AddWithValue("@Rating", int.Parse(txtRating.Text));
    cmd.Parameters.AddWithValue("@Checked_By", int.Parse(txtCheckedBy.Text));
    cmd.Parameters.AddWithValue("@Check_Report", txtCheckReport.Text);

    con.Open();
    int rowsAffected = cmd.ExecuteNonQuery();

    MessageBox.Show(rowsAffected > 0 ? "Review updated successfully." : "Review not found.");
    LoadTable("Reviews");
}
}
catch (Exception ex)
{
    MessageBox.Show("Error updating review: " + ex.Message);
}
}

```

Explanation:

- This method updates an existing review record when the update button is clicked
- It follows the standard pattern of using a stored procedure with parameters drawn from form fields
- Multiple `int.Parse()` calls are used for numeric fields without validation, creating potential failure points:
 - `@ReviewID` - the primary key for the record being updated
 - `@RentalID` - a foreign key to the associated rental
 - `@Rating` - should be a number between 1-5 (as defined in the database schema)
 - `@Checked_By` - a foreign key to the employee who verified the review
- The `ExecuteNonQuery()` method is appropriate for UPDATE operations and returns the number of rows affected
- The conditional operator (`?:`) in the message box provides different feedback based on the update result:
 - If rows affected > 0, the update succeeded
 - If rows affected = 0, no matching review was found (based on ReviewID)
- This approach provides immediate feedback on operation success or failure
- If an exception occurs during the update (e.g., from invalid input), a specific error message is shown
- After updating, the display is refreshed with `LoadTable("Reviews")` regardless of success or failure
- The error message specifically identifies the operation that failed ("Error updating review"), making troubleshooting easier

The ReviewsControl demonstrates consistent error handling and user feedback across all database operations.

2.9. VEHICLE CONTROL

The VehicleControl manages vehicle information and demonstrates more extensive validation and complex form interaction.

	CarID	Brand	Type	Model_Name	Model_Year	Color	Availability_Status	Number_of_Seats	License_Number	License_Expiry_Da	GarageID
▶	1	Honda	Sedan	Civic	2021	Red	yes	5	ABC123	12/31/2026	1
	2	Toyota	SUV	Corolla	2022	Blue	no	7	XYZ789	8/15/2027	2
	3	BMW	Coupe	M4	2020	Black	yes	4	BMW987	11/25/2025	1
	4	Ford	Truck	F-150	2019	White	yes	2	FRE001	3/1/2026	2
	5	Chevrolet	Sedan	Malibu	2023	Silver	no	5	CHE123	6/10/2027	1
*											

Input Validation:

```
if (!string.IsNullOrEmpty(txtCarID.Text))
{
    MessageBox.Show("You cannot enter CarID manually. It is auto-generated.");
    return;
}

if (string.IsNullOrEmpty(txtBrand.Text) ||
    string.IsNullOrEmpty(txtModelName.Text) ||
    string.IsNullOrEmpty(txtModelYear.Text) ||
    string.IsNullOrEmpty(txtType.Text) ||
    string.IsNullOrEmpty(txtColor.Text) ||
    string.IsNullOrEmpty(txtNumberOfSeats.Text) ||
    string.IsNullOrEmpty(txtLicenseNumber.Text) ||
    string.IsNullOrEmpty(txtGarageID.Text) ||
    string.IsNullOrEmpty(txtStatus.Text))
{
    MessageBox.Show("All fields must be filled.");
    return;
}

if (!int.TryParse(txtModelYear.Text, out int modelYear) ||
    !int.TryParse(txtNumberOfSeats.Text, out int seats) ||
    !int.TryParse(txtGarageID.Text, out int garageId))
{
    MessageBox.Show("Please enter valid numeric and date values.");
    return;
}

int currentYear = DateTime.Now.Year;
```

```

if (modelYear < 1950 || modelYear > currentYear)
{
    MessageBox.Show($"Model year must be between 1950 and {currentYear}.");
    return;
}

if (!System.Text.RegularExpressions.Regex.IsMatch(txtLicenseNumber.Text, @"^\d{5}$"))
{
    MessageBox.Show("License Number must be exactly 5 digits.");
    return;
}

string status = txtStatus.Text.Trim().ToLower();
if (status != "yes" && status != "no")
{
    MessageBox.Show("Availability Status must be either 'yes' or 'no'.");
    return;
}

```

Explanation:

- This comprehensive validation block implements multiple layers of data validation:
 1. **Primary key protection:** Prevents manual entry of auto-generated IDs
 2. **Required field validation:** Checks that all necessary fields are completed
 3. **Data type validation:** Ensures numeric fields contain valid numbers
 4. **Range validation:** Confirms values fall within acceptable ranges
 5. **Format validation:** Enforces specific patterns for formatted fields
 6. **Value set validation:** Ensures values are from valid sets
- The validation uses early returns as guard clauses, stopping execution at the first validation failure
- Each validation is accompanied by a specific, descriptive error message that guides the user
- Notable validation techniques include:
 - `string.IsNullOrEmpty()` to check for empty, null, or whitespace-only input
 - `int.TryParse()` for safe numeric conversion without exceptions
 - `DateTime.Now.Year` to get the current year for range checking
 - `Regex.IsMatch()` with pattern `@"^\d{5}$"` to validate license number format
 - `Trim().ToLower()` to normalize input for case-insensitive comparison
- The validation uses modern C# features such as:
 - Out variables declared inline (`out int modelYear`)
 - String interpolation (`$"Model year must be between 1950 and {currentYear}."`)
 - Verbatim string literals for regex patterns (`@"^\d{5}$"`)
- This approach enforces business rules at the application level before data reaches the database

Searching for Vehicles:

```

private void btnSearch_Click(object sender, EventArgs e)
{
    if (!int.TryParse(txtCarID.Text, out int carId))
    {
        MessageBox.Show("Please enter valid numeric and date values.");
        return;
    }
}

```

```

try
{
    using (SqlConnection con = new SqlConnection(connectionString))
    {
        string query = "SELECT * FROM Vehicle WHERE CarID = @CarID";
        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@CarID", carId);
        con.Open();
        SqlDataReader reader = cmd.ExecuteReader();

        if (reader.Read())
        {
            txtCarID.Text = reader["CarID"].ToString();
            originalCarId = Convert.ToInt32(reader["CarID"]);
            txtBrand.Text = reader["Brand"].ToString();
            txtModelName.Text = reader["Model_Name"].ToString();
            txtModelYear.Text = reader["Model_Year"].ToString();
            txtType.Text = reader["Type"].ToString();
            txtColor.Text = reader["Color"].ToString();
            txtNumberOfSeats.Text = reader["Number_of_Seats"].ToString();
            dtpLicenseExpiry.Text = reader["License_Expiry_Date"].ToString();
            txtLicenseNumber.Text = reader["License_Number"].ToString();
            txtGarageID.Text = reader["GarageID"].ToString();
            txtStatus.Text = reader["Availability_Status"].ToString();
        }
        else
        {
            MessageBox.Show("Vehicle not found.");
        }

        con.Close();
    }
}
catch (Exception ex)
{
    MessageBox.Show("An error occurred while searching vehicle: " + ex.Message);
}
}

```

Explanation:

- This method implements a search feature based on vehicle ID and populates the form with the found record
- The search process follows these steps:
 1. Validate that the car ID is a valid integer
 2. Establish a database connection
 3. Create a parameterized query to find the vehicle by ID
 4. Execute the query and attempt to read the result
 5. If found, populate all form fields with vehicle data
 6. If not found, display a “not found” message
- The code uses parameterized queries (**@CarID**) to prevent SQL injection attacks

- The `SqlDataReader` is used to efficiently retrieve a single record
- The `if (reader.Read())` condition checks if any record was returned before attempting to access data
- The `originalCarId` variable (likely a class field) stores the ID for future reference, possibly for update operations
- Each form field is mapped to its corresponding database column using column name indexers
- The explicit `con.Close()` call is redundant since the connection is in a `using` block, but provides defensive programming
- The error message is specific to the search operation, making troubleshooting easier
- This code could be improved by:
 - Disposing the reader explicitly
 - Validating that the car ID field is not empty before attempting conversion

The VehicleControl demonstrates thorough input validation, including range checks, pattern matching, and required field validation, ensuring data integrity at the application level before it reaches the database.

2.10. VEHICLEMAINTENANCE CONTROL

The VehicleMaintenanceControl manages the many-to-many relationship between vehicles and maintenance records.

The screenshot shows a web application dashboard with a tabbed interface. The 'Vehicle Needs Maintenance' tab is active. It contains two input fields: 'Maintenance Number' and 'Car ID'. Below these fields are three buttons: 'Insert', 'Update', and 'Delete'. At the bottom, there is a data grid with two columns: 'Maintenance_Num1' and 'CarID'. The grid contains three rows of data, with the first row (1, 1) selected.

	Maintenance_Num1	CarID
▶	1	1
	2	1
	1	2
*		

Updating Vehicle Maintenance Records:

```
private void vehicleUpdateButton_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a record to update.");
            return;
        }

        int oldMaintenanceNumber = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells
["Maintenance_Number"].Value);
        int oldCarID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["CarID"].Va
lue);
```

```

using (SqlConnection con = new SqlConnection(conn.ConnectionString))
{
    SqlCommand cmd = new SqlCommand("UpdateVehicleNeedsMaintenance", con)
    {
        CommandType = CommandType.StoredProcedure
    };

    cmd.Parameters.AddWithValue("@Old_Maintenance_Number", oldMaintenanceNumber);

    cmd.Parameters.AddWithValue("@Old_CarID", oldCarID);
    cmd.Parameters.AddWithValue("@New_Maintenance_Number", int.Parse(maintenanceTextBox.Text));
    cmd.Parameters.AddWithValue("@New_CarID", int.Parse(carTextBox.Text));

    con.Open();
    int affectedRows = cmd.ExecuteNonQuery();

    MessageBox.Show(affectedRows > 0 ? "Record updated successfully." : "Record not found.");
    LoadTable("Vehicle_Needs_Maintenance");
}
}
catch (Exception ex)
{
    MessageBox.Show("Error updating vehicle maintenance record: " + ex.Message);
}
}

```

Explanation:

- This method demonstrates update handling for a many-to-many relationship table (junction table)
- The update process for junction tables differs from standard entity updates:
 1. First, existing relationship values are retrieved (**oldMaintenanceNumber, oldCarID**)
 2. Then both old and new values are passed to the update procedure
 3. The stored procedure typically deletes the old record and inserts a new one
- The code follows this sequence:
 1. Verify a row is selected for update
 2. Extract the current composite key values from the selected grid row
 3. Establish a database connection
 4. Configure the update stored procedure with old and new values
 5. Execute the update and check the result
 6. Refresh the grid to show the updated data
- The method accesses grid cells by name (**Cells["Maintenance_Number"]**) rather than by index for better clarity
- Unsafe conversion methods are used:
 - **Convert.ToInt32()** for grid values, which could throw exceptions if the cell is null or contains non-numeric data
 - **int.Parse()** for text box values, which could throw exceptions if the input is invalid
- The method assumes a specific grid structure with columns named "Maintenance_Number" and "CarID"
- The update procedure requires four parameters:

- **@Old_Maintenance_Number** and **@Old_CarID** to identify the existing record
- **@New_Maintenance_Number** and **@New_CarID** for the updated relationship
- This pattern is necessary for junction tables where the primary key is a composite of foreign keys

The VehicleMaintenanceControl demonstrates handling of many-to-many relationships in the user interface, allowing users to associate vehicles with maintenance records.

2.11. VEHICLERENTAL CONTROL

The VehicleRentalControl manages the many-to-many relationship between vehicles and rentals, including pricing information.

The screenshot shows a web application dashboard titled "Dashboard" with several tabs: Customer, Rental, Payment, Maintenance, Garage, Vehicle, Vehicle Offered For Rental (selected), Vehicle Needs Maintenance, Employee, and Reviews. Below the tabs, there's a section titled "Vehicles offered for Rental" containing three input fields: "CarID" with value "1", "RentalID" with value "1", and "Daily_price" with value "23". Below these fields are four buttons: "Search", "Insert" (highlighted), "Update", and "Delete". At the bottom, there's a table with columns "RentalID", "CarID", and "Daily_Price". The first row shows "1", "1", and "23.00" respectively, and is highlighted in blue. A second row with asterisks indicates a new entry.

	RentalID	CarID	Daily_Price
▶	1	1	23.00
*			

Searching for Vehicle Rentals:

```
private void btnSearch_Click(object sender, EventArgs e)
{
    try
    {
        string query = "SELECT * FROM Vehicle_Offered_For_Rental WHERE 1=1";
        List<SqlParameter> parameters = new List<SqlParameter>();

        if (int.TryParse(txtCarID.Text, out int carId))
        {
            query += " AND CarID = @CarID";
            parameters.Add(new SqlParameter("@CarID", carId));
        }

        if (int.TryParse(txtRentalID.Text, out int rentalId))
        {
            query += " AND RentalID = @RentalID";
            parameters.Add(new SqlParameter("@RentalID", rentalId));
        }

        if (parameters.Count == 0)
```

```

    {
        MessageBox.Show("Please enter at least CarID or RentalID to search.");
        return;
    }
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddRange(parameters.ToArray());
        SqlDataAdapter adapter = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        adapter.Fill(dt);
        dataGridView1.DataSource = dt;
    }
}
catch (Exception ex)
{
    MessageBox.Show("Error while searching: " + ex.Message);
}
}

```

Explanation:

- This method implements a dynamic search feature that builds the query based on user-provided criteria
- The code demonstrates an advanced dynamic SQL technique with several key features:
 1. Starting with a base query containing **WHERE 1=1** (always true) to simplify conditional clause addition
 2. Building a parameter collection in parallel with the query string
 3. Adding conditions only when valid search criteria are provided
 4. Using parameterized queries to prevent SQL injection
- The search logic works as follows:
 1. Initialize the base query and an empty parameter list
 2. For each search field, validate and conditionally add to the query and parameter list
 3. Check if at least one search parameter was provided
 4. Execute the query with all parameters and display results
- The code uses **int.TryParse()** for safe conversion of ID fields
- The **List<SqlParameter>** collection stores all parameters for the query
- Parameters are added to the command at once with **cmd.Parameters.AddRange(parameters.ToArray())**
- This approach supports both single-criterion and multi-criteria searches
- The clause **WHERE 1=1** is a common pattern that allows adding conditions with AND without special handling for the first condition
- Using SqlDataAdapter and DataTable provides a clean way to fill the grid with filtered results
- The method requires at least one valid search parameter, preventing full table retrieval
- This pattern could be extended to support additional search criteria with minimal changes

The VehicleRentalControl demonstrates dynamic SQL query building based on search criteria, showing how to create flexible search functionality with proper parameterization to prevent SQL injection.

2.12. DASHBOARD OVERVIEW

The Dashboard serves as the main interface that integrates all the individual controls into a cohesive application using a tab-based interface.

Dashboard Implementation:

```
public partial class Dashboard : Form
{
    public Dashboard()
    {
        InitializeComponent();
        this.Load += DashboardForm_Load;
    }

    private void DashboardForm_Load(object sender, EventArgs e)
    {
        var customerControl = new CustomerControl();
        customerControl.Dock = DockStyle.Fill;
        tabCustomer.Controls.Clear();
        tabCustomer.Controls.Add(customerControl);

        var rentalControl = new RentalsControl();
        rentalControl.Dock = DockStyle.Fill;
        tabRental.Controls.Clear();
        tabRental.Controls.Add(rentalControl);

        // ... Additional controls loaded
    }
}
```

Integrated Controls:

- **CustomerControl:** Manages customer information and profiles
- **RentalsControl:** Handles rental bookings and transactions
- **PaymentControl:** Processes and tracks payments
- **MaintenanceControl:** Records and schedules vehicle maintenance
- **GarageControl:** Manages garage information and employee assignments
- **VehicleControl:** Handles vehicle inventory and details
- **VehicleRentalControl:** Manages the relationship between vehicles and rentals
- **VehicleMaintenanceControl:** Tracks which vehicles need maintenance
- **EmployeeControl:** Manages employee information
- **ReviewsControl:** Handles customer feedback and reviews

Centralized Architecture Benefits:

1. **Unified User Experience:** Users access all functionality through a single interface with consistent navigation
2. **Modular Design:** Each functional area is encapsulated in its own control, enabling focused development and maintenance
3. **Separation of Concerns:** Business logic for each entity is isolated within its specific control
4. **Simplified Navigation:** The tab-based interface allows users to quickly switch between different system functions
5. **Consistent Visual Design:** The common container ensures a uniform appearance across all functional areas

The Dashboard serves as the glue that binds individual components together while maintaining their independence, demonstrating good architectural design principles and improving the overall user experience.

2.13. SUMMARY OF USER INTERFACE IMPLEMENTATION

The system is implemented as a Windows Forms application using C# and ADO.NET for database connectivity. Each entity in the database has a corresponding user control that provides a graphical interface for CRUD (Create, Read, Update, Delete) operations on that entity. The Dashboard integrates all these controls into a cohesive user experience with consistent navigation and visual design.

3. ERROR HANDLING AND DATA VALIDATION

The application implements comprehensive error handling and data validation to ensure data integrity and application stability. The following error handling strategies are evident throughout the codebase:

3.1. DATABASE CONNECTION ERROR HANDLING

- **Connection Initialization and State Checking:** The GarageControl checks connection status before performing operations:

```
if (con == null) {  
    MessageBox.Show("Database connection is not initialized.");  
    return;  
}
```

Explanation:

- This defensive programming technique prevents null reference exceptions by checking connection state before use
 - The check happens at the beginning of database-related methods, serving as a guard clause
 - If the connection is null, a user-friendly message is displayed explaining the issue
 - The **return** statement immediately exits the method, preventing execution of potentially problematic code
 - This pattern centralizes connection validation rather than repeating similar checks throughout the code
 - The check doesn't verify if the connection is open, only that it exists, suggesting the open operation happens elsewhere
- **Connection Resource Management:** All database operations use **using** statements to ensure connections are properly closed, even if exceptions occur:

```
using (SqlConnection con = new SqlConnection(conn.ConnectionString)) {  
    // Database operations  
}
```

Explanation:

- The **using** statement implements the Dispose pattern, automatically calling the **Dispose()** method when execution leaves the block
- For SqlConnection, the Dispose method ensures the connection is properly closed and resources are released
- This prevents connection leaks, which can deplete the connection pool and cause performance issues
- The pattern guarantees cleanup even if exceptions occur within the block
- Unlike manually calling **Open()** and **Close()**, this approach is exception-safe
- Each connection is scoped to the operation that needs it, improving resource utilization
- This is an example of the RAII (Resource Acquisition Is Initialization) pattern from C++, adapted for C#

3.2. EXCEPTION HANDLING FOR DATABASE OPERATIONS

- **Try-Catch Blocks:** All database operations are wrapped in try-catch blocks to prevent application crashes:

```
try {  
    // Database operations  
}  
catch (Exception ex) {  
    MessageBox.Show("Error: " + ex.Message);  
}
```

Explanation:

- This pattern is a fundamental error handling technique that:
 1. Attempts to execute potentially error-prone code in the **try** block
 2. Catches any exceptions that occur in the **catch** block
 3. Provides user feedback rather than allowing the application to crash
 - The generic **Exception** type catches all exception types, making it a comprehensive safety net
 - The error message displays the exception's **Message** property, which contains a human-readable description
 - The **MessageBox.Show()** method creates a modal dialog that:
 - Informs the user that an error occurred
 - Requires user acknowledgment before continuing
 - Allows the application to continue running despite the error
 - This approach provides graceful error handling but lacks:
 - Logging of the exception for future troubleshooting
 - Different handling strategies for different exception types
 - Recovery mechanisms beyond simple user notification
 - The pattern follows defensive programming principles by anticipating and handling potential failures
- **Specific SQL Exception Handling:** Some operations distinguish between general exceptions and SQL-specific exceptions:

```
catch (SQLException ex) {  
    // sql error, y3ni feh moshkela fel query aw constraints  
    MessageBox.Show("DB Error: " + ex.Message, "SQL Error", MessageBoxButtons.OK  
    , MessageBoxIcon.Error);  
}  
catch (Exception ex) {  
    // ay error tanya  
    MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, Messa  
geBoxIcon.Error);  
}
```

Explanation:

- This more sophisticated exception handling pattern uses multiple catch blocks to handle different exception types differently
- The exception types are arranged from most specific to most general, following C# best practices:
 1. **SQLException** - Catches database-specific errors (constraint violations, timeouts, etc.)
 2. **Exception** - Catches any other exceptions that might occur
- The SQLException handler:
 - Provides a "DB Error" prefix to clearly indicate a database-related issue

- Sets the dialog title to “SQL Error” for clear categorization
- Uses the Error icon to visually indicate a more serious problem
- Includes a transliterated Arabic comment explaining that the error relates to query or constraint issues
- The general Exception handler:
 - Handles all other exception types with a generic “Error” prefix
 - Sets the dialog title to “Error”
 - Also uses the Error icon
 - Has a comment in transliterated Arabic noting that this catches “any other error”
- This approach allows for:
 - Different user messaging based on error type
 - Potential different recovery strategies (not shown in this example)
 - Better problem categorization for troubleshooting
 - More specific guidance to users based on error type

3.3. DATA TYPE VALIDATION

- **Numeric Input Validation:** All forms validate numeric inputs before processing:

```
if (!int.TryParse(txtAge.Text, out int age) || age < 0) {
    MessageBox.Show("Age must be a valid positive number.", "Validation Error",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}
```

Explanation:

- This validation combines parsing and range checking in a single condition for efficiency
- The code demonstrates modern C# features with inline variable declaration in the `TryParse` method
- The validation consists of two parts combined with the logical OR operator (`||`):
 1. `!int.TryParse(txtAge.Text, out int age)` - Checks if the input can be converted to an integer
 2. `age < 0` - Checks if the parsed value is negative
- If either condition is true, the validation fails:
 - Either the input is not a valid integer
 - Or the input is a negative number
- The `TryParse` method is safer than `Parse` because it:
 - Returns a boolean success/failure result
 - Outputs the parsed value via an out parameter
 - Never throws an exception on invalid input
- The error message is specific to the exact validation failure
- The `MessageBox` includes:
 - A descriptive message
 - A categorized title (“Validation Error”)
 - A warning icon
- The early `return` statement prevents further processing with invalid data
- **DBNull Handling:** When reading from database, the code checks for DBNull values:

```
row["Employee ID"] = reader["EmployeeID"] != DBNull.Value ? Convert.ToInt32(read
er["EmployeeID"]) : 0;
```

Explanation:

- This code demonstrates proper handling of NULL values from database results
- In ADO.NET, database NULL values are represented as `DBNull.Value`, not C# `null`
- The ternary conditional operator (`? :`) provides compact if-else logic:
 - If the database field is not `DBNull`, convert the value to an integer
 - If the database field is `DBNull`, use 0 as a default value
- This approach prevents potential `InvalidCastException` errors when attempting to convert `DBNull` to numeric types
- The pattern substitutes a meaningful default value (0) rather than propagating null, which simplifies further processing
- This is especially important when populating UI elements like `DataGridView` cells that may not handle null values well
- The explicit conversion using `Convert.ToInt32()` handles various numeric types from the database
- This pattern could be adapted for other data types with appropriate default values:
 - Empty strings for text
 - `DateTime.MinValue` for dates
 - `False` for boolean values

3.4. INPUT FORMAT VALIDATION

- **Regular Expression Validation:** Text inputs are validated using regex where specific formats are required:

```
if (!Regex.IsMatch(txtName.Text, @"^[a-zA-Z ]+$")) {  
    MessageBox.Show("Name must contain only letters and spaces.", "Validation Error",  
        MessageBoxButtons.OK, MessageBoxIcon.Warning);  
    return;  
}
```

Explanation:

- This code demonstrates pattern matching validation using regular expressions from the `System.Text.RegularExpressions` namespace
- The regular expression pattern `^[a-zA-Z ]+$` is used to validate the format of a name field:
 - `^` asserts the start of the string
 - `[a-zA-Z ]` defines a character class that allows:
 - Any lowercase letter (a-z)
 - Any uppercase letter (A-Z)
 - A space character
 - `+` quantifier requires one or more of these characters
 - `$` asserts the end of the string
- Combined, this pattern ensures the entire string contains only letters and spaces with no other characters
- The pattern is preceded by `@` (verbatim string literal) to avoid escaping backslashes in the regular expression
- The `!` operator negates the result, so the condition evaluates to true when the pattern does NOT match
- When validation fails, a specific error message explains exactly what format is expected
- The `MessageBox.Show()` method displays the error with:
 - A descriptive message
 - A categorized title ("Validation Error")
 - A warning icon
- The early `return` statement prevents further processing with invalid data

- **License Number Format:** Vehicle license numbers are validated:

```
if (!System.Text.RegularExpressions.Regex.IsMatch(txtLicenseNumber.Text, @"^\d{5}$")) {
    MessageBox.Show("License Number must be exactly 5 digits.");
    return;
}
```

Explanation:

- This validation ensures license numbers conform to a specific format requirement
- The fully qualified class name **System.Text.RegularExpressions.Regex** is used, suggesting either:
 - The **using System.Text.RegularExpressions;** directive is missing, or
 - The developer chose to use the full namespace for clarity
- The regular expression pattern **^\d{5}\$** enforces a strict 5-digit format:
 - **^** asserts the start of the string
 - **\d** matches any digit (0-9)
 - **{5}** specifies exactly 5 occurrences of the preceding element
 - **\$** asserts the end of the string
- This ensures the license number:
 - Contains only digits
 - Has exactly 5 digits (no more, no less)
 - Has no spaces or other characters
- The error message clearly communicates the expected format
- Unlike the name validation example, this message box doesn't specify a title or icon, using defaults instead
- This validation would reject formats like:
 - "ABC123" (contains letters)
 - "123" (too few digits)
 - "123456" (too many digits)
 - "12 345" (contains a space)

3.5. REQUIRED FIELD VALIDATION

- **Empty Field Checking:** Forms check that all required fields are filled:

```
if (string.IsNullOrEmpty(txtBrand.Text) ||
    string.IsNullOrEmpty(txtModelName.Text) ||
    // Additional fields
) {
    MessageBox.Show("All fields must be filled.");
    return;
}
```

Explanation:

- This code implements comprehensive validation for required fields to prevent incomplete data submission
- The validation uses the **string.IsNullOrEmpty()** method, which checks for three conditions:
 1. **null** values (reference is null)
 2. Empty strings ("")
 3. Whitespace-only strings (spaces, tabs, etc.)

- Multiple field checks are combined with the logical OR operator (`||`):
 - If ANY of the conditions evaluates to true, the entire expression is true
 - This means if ANY field is empty, the validation fails
- This approach is more thorough than `string.IsNullOrEmpty()` because it also catches strings with only whitespace
- The error message is generic ("All fields must be filled") rather than specifying which field is missing
- After displaying the error message, the `return` statement immediately exits the method
- This validation pattern ensures data completeness, a fundamental aspect of data integrity
- In a more sophisticated implementation, this might be enhanced to:
 - Identify which specific field is missing
 - Visually highlight the problematic field(s)
 - Handle optional fields separately from required ones

3.6. RANGE AND VALUE VALIDATION

- **Date Range Validation:** Ensures dates fall within valid ranges:

```
int currentYear = DateTime.Now.Year;
if (modelYear < 1950 || modelYear > currentYear) {
    MessageBox.Show($"Model year must be between 1950 and {currentYear}.");
    return;
}
```

Explanation:

- This validation enforces business rules about acceptable date ranges for vehicle model years
 - The code first gets the current year using `DateTime.Now.Year` as the upper boundary for validation
 - The validation logic enforces that model years must be:
 1. Not older than 1950 (preventing antique or very old vehicles)
 2. Not newer than the current year (preventing future model years)
 - The condition uses the logical OR (`||`) to fail if EITHER boundary is violated
 - If the validation fails, a descriptive error message is shown that includes:
 - The expected minimum value (1950)
 - The expected maximum value (current year)
 - String interpolation (`$"..."`) allows embedding the dynamic current year value directly in the message
 - This pattern demonstrates dynamic boundary validation based on current date
 - The fixed lower boundary (1950) suggests a business rule about the oldest vehicles accepted by the system
 - Similar range validations could be used for other date or numeric fields:
 - Rental periods (must be within reasonable duration)
 - Ages (must be within legal limits)
 - Prices (must be within market expectations)
- **Value Set Validation:** Ensures values are from valid sets:

```
string status = txtStatus.Text.Trim().ToLower();
if (status != "yes" && status != "no") {
    MessageBox.Show("Availability Status must be either 'yes' or 'no'.");
    return;
}
```

Explanation:

- This validation ensures input conforms to a predefined set of allowed values
- The validation process includes pre-processing steps to normalize the input:
 1. `Trim()` removes leading and trailing whitespace
 2. `ToLower()` converts the text to lowercase
- These normalization steps allow for more flexible input recognition:
 - “Yes”, “YES”, “yes”, “ yes “ will all be accepted
- The validation uses a logical AND (&&) with negation to check for invalid values:
 - If the value is not “yes” AND not “no”, it’s invalid
- The error message clearly states the only acceptable values
- Ideally, this type of validation would be implemented with a dropdown (ComboBox) to prevent invalid input
- This pattern is commonly used for fields with a limited set of valid values such as:
 - Status fields (Active/Inactive, Enabled/Disabled)
 - Boolean-like fields (Yes/No, True/False)
 - Category selections (Fixed list of categories)
- A more sophisticated approach would use enumerations or lookup tables for validation

3.7. UI STATE VALIDATION

- **Selection Validation:** Ensures a row is selected before operations that require it:

```
if (dataGridView.SelectedRows.Count == 0) {
    MessageBox.Show("Select a row to update.", "No Selection", MessageBoxButtons
.OK, MessageBoxIcon.Warning);
    return;
}
```

Explanation:

- This validation prevents operations that require a selected row when no selection exists
 - The check examines the `SelectedRows` collection’s `Count` property:
 - If zero, no rows are selected and the operation can’t proceed
 - If greater than zero, at least one row is selected
 - When validation fails, the error message:
 1. Clearly explains what action is needed (“Select a row to update”)
 2. Specifies the error type in the title (“No Selection”)
 3. Uses a warning icon to differentiate from more serious errors
 - This validation is essential for operations like:
 - Updates (need to know which record to update)
 - Deletions (need to know which record to delete)
 - Detail views (need to know which record to display)
 - This is a form of state validation rather than data validation
 - It’s positioned before any business logic to fail fast if prerequisites aren’t met
 - This pattern prevents `NullReferenceExceptions` and provides better user guidance than a crash
 - It follows the principle of defensive programming by anticipating potential error conditions
- **Auto-filled Form Fields:** Reduces user error by automatically filling form fields from selected grid rows:

```
private void FillFieldsFromGrid() {
    if (dataGridView.SelectedRows.Count == 0) return;
    var row = dataGridView.SelectedRows[0];
    txtMaintenanceNumber.Text = row.Cells[0].Value.ToString();
}
```

```
} // Additional field assignments
```

Explanation:

- This method implements an automatic data-binding pattern to synchronize grid selections with form fields
- The implementation follows these steps:
 1. Guard against no selection with an early return
 2. Get the first selected row (even if multiple rows are selected)
 3. Extract values from each cell and assign to corresponding form fields
- This pattern reduces user error by:
 - Eliminating manual retyping of data
 - Ensuring exact data transfer from grid to form fields
 - Maintaining data consistency between display and edit views
- Benefits of this approach include:
 - Improved user experience through reduced data entry
 - Reduced risk of transcription errors
 - Faster workflow for data editing
 - Immediate visual feedback on selection
- This is often triggered by selection change events to keep the form continuously updated
- In a Model-View-Controller pattern, this represents synchronization between different views of the same data
- This type of binding is manually implemented here, but more sophisticated frameworks offer declarative data binding

3.8. USER FEEDBACK

- **Categorized Error Messages:** Provides clearly categorized error messages:

```
MessageBox.Show("Name must contain only letters and spaces.", "Validation Error",  
    MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

Explanation:

- This code demonstrates a comprehensive message box that provides well-structured user feedback
- The message box includes several key components:
 1. A specific, action-oriented error message explaining exactly what's wrong
 2. A categorized title ("Validation Error") that classifies the type of issue
 3. An appropriate icon (Warning) that visually indicates severity
 4. A single button (OK) that acknowledges the message
- The message text is clear and instructs the user about:
 - What the problem is (invalid characters in name)
 - What the correct format should be (only letters and spaces)
- The title "Validation Error" categorizes this as input validation rather than a system error
- The warning icon (yellow triangle with exclamation mark) indicates:
 - This is important but not critical
 - The user needs to take action
 - The application is still functioning properly
- This level of detail helps users:
 - Understand exactly what went wrong
 - Know precisely how to fix it

- Distinguish between different types of errors
 - This pattern is part of a larger error handling strategy that prioritizes user experience

- **Operation Result Feedback:** Informs users about the success or failure of operations:

```
MessageBox.Show(rowsAffected > 0 ? "Review updated successfully." : "Review not found.");
```

Explanation:

- This code provides dynamic feedback based on the actual outcome of a database operation
- The implementation uses the conditional (ternary) operator (`? :`) to select between two messages:
 - If `rowsAffected > 0`: "Review updated successfully."
 - If `rowsAffected = 0`: "Review not found."
- This approach leverages the return value from `ExecuteNonQuery()` which indicates:
 - For UPDATE operations: the number of rows affected
 - For INSERT operations: the number of rows inserted
 - For DELETE operations: the number of rows deleted
- The pattern distinguishes between two distinct scenarios:
 1. Success - The update was performed on the specified record
 2. Failure due to record not found - The specified record doesn't exist
- This gives users precise information about what happened without technical details
- This is more user-friendly than a generic "Operation completed" message
- It allows users to take appropriate next steps based on the actual outcome
- This pattern could be enhanced by adding visual cues (different icons for success/failure)

- **Technical Details for Troubleshooting:** Includes exception details in error messages:

```
MessageBox.Show("Error loading employees: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
```

Explanation:

- This code provides more technical error feedback for troubleshooting purposes
- The message combines:
 1. A context-specific prefix ("Error loading employees: ") that identifies the operation
 2. The actual exception message (`ex.Message`) containing technical details
- The dialog is configured with:
 - A clear title ("Error") indicating a problem occurred
 - An error icon (red circle with X) signaling that something went wrong
 - A single OK button for acknowledgment
- This approach balances usability with troubleshooting capability:
 - Users understand what operation failed
 - Support personnel get technical details for diagnosis
- Including the exception message helps identify:
 - Database connection issues
 - SQL syntax errors
 - Access permission problems
 - Constraint violations
- In production environments, this might be replaced with:
 - A user-friendly message for display
 - Background logging of full exception details

- Error reporting to a monitoring system
- This pattern represents basic error reporting suitable for development or internal applications

The consistent implementation of these error handling and validation strategies throughout the application ensures data integrity and provides a more robust user experience by preventing application crashes and data inconsistencies.

4. STORED PROCEDURES IMPLEMENTATION

The system implements stored procedures for all database operations to encapsulate business logic, improve security, and maintain consistency. These procedures are organized by entity type and operation, with consistent patterns across the system.

4.1. INSERT PROCEDURES

Insert procedures are used to add new records to the database. They take entity attributes as parameters and return the newly created ID using `SCOPE_IDENTITY()`.

EXAMPLE: INSERTCUSTOMER.SQL

```
CREATE PROCEDURE InsertCustomer
    @Customer_Name VARCHAR(100),
    @Address VARCHAR(255),
    @Date_of_Birth DATE,
    @Age INT,
    @Email VARCHAR(100),
    @Phone_Number VARCHAR(20),
    @Password VARCHAR(100),
    @License_Number VARCHAR(50),
    @License_Expiry_Date DATE,
    @License_Personal_Photo VARCHAR(MAX)
AS
BEGIN
INSERT INTO Customer (Customer_Name, Address, Date_of_Birth, Age, Email, Phone_Number,
    Password, License_Number, License_Expiry_Date, License_Personal_Photo)
VALUES (@Customer_Name, @Address, @Date_of_Birth, @Age, @Email, @Phone_Number, @Passwo
rd, @License_Number, @License_Expiry_Date, @License_Personal_Photo);

SELECT SCOPE_IDENTITY(); -- return new CustomerID
END
```

VARIATIONS IN INSERT PROCEDURES

Each entity has its own insert procedure with parameters specific to its table schema:

- **InsertEmployee.sql:** Handles simpler data with fewer parameters (Name, Role, Age, Salary, Enrollment_Date).
- **InsertMaintenance.sql:** Focuses on service data (Service_Price, Company_Name, Date).
- **InsertPayment.sql:** Contains parameters for financial transactions (Total_Price, Payment_Status, Payment_Date, CustomerID, RentalID).
- **InsertRental.sql:** Manages rental bookings with date parameters (StartDate, EndDate, Status, CustomerID).
- **InsertReview.sql:** Contains feedback parameters (RentalID, Customer_Name, Feedback, Rating, Checked_By, Check_Report).
- **InsertVehicle.sql:** Handles complex vehicle information with structured formatting for clarity.

JUNCTION TABLE INSERT PROCEDURES

Junction tables have simpler insert procedures that handle composite keys:

- **InsertVehicleNeedsMaintenance.sql**: Links vehicles to maintenance records with just two parameters (Maintenance_Number, CarID).
- **InsertVehicleRental.sql**: Connects vehicles to rentals with pricing information (CarID, RentalID, Daily_price).

4.2. UPDATE PROCEDURES

Update procedures modify existing records using primary keys to target specific rows.

EXAMPLE: UPDATEEMPLOYEE.SQL

```
CREATE PROCEDURE UpdateEmployee
```

```
    @EmployeeID INT,  
    @Name VARCHAR(100),  
    @Role VARCHAR(50),  
    @Age INT,  
    @Salary DECIMAL(10,2),  
    @Enrollment_Date DATE
```

```
AS
```

```
BEGIN
```

```
    UPDATE Employee
```

```
    SET Name = @Name, Role = @Role, Age = @Age, Salary = @Salary, Enrollment_Date = @E
```

```
nrollment_Date
```

```
    WHERE EmployeeID = @EmployeeID
```

```
END
```

VARIATIONS IN UPDATE PROCEDURES

Update procedures follow a consistent pattern but differ based on the entity's attributes:

- **UpdateCustomer.sql**: Contains the most parameters, including personal and license information.
- **UpdateMaintenance.sql**: Updates service records with minimal fields (Service_Price, Company_Name, Date).
- **UpdatePayment.sql**: Modifies financial transaction details and their associations.
- **UpdateRental.sql**: Updates booking dates and status information.
- **UpdateReview.sql**: Manages customer feedback and employee verification information.
- **UpdateVehicle.sql**: Contains the most detailed vehicle information parameters and requires structured formatting.

JUNCTION TABLE UPDATE PROCEDURES

Junction table updates have unique patterns:

- **UpdateVehicleNeedsMaintenance.sql**: Uses a delete-then-insert pattern with both old and new composite key values:

```
CREATE PROCEDURE UpdateVehicleNeedsMaintenance
```

```
    @Old_Maintenance_Number INT,  
    @Old_CarID INT,  
    @New_Maintenance_Number INT,  
    @New_CarID INT
```

```
AS
```

```
BEGIN
```

```

DELETE FROM Vehicle_Needs_Maintenance
WHERE Maintenance_Number = @Old_Maintenance_Number AND CarID = @Old_CarID;

INSERT INTO Vehicle_Needs_Maintenance (Maintenance_Number, CarID)
VALUES (@New_Maintenance_Number, @New_CarID);
END

```

- **UpdateVehicleRental.sql**: Simplifies the update to just modify the Daily_price while maintaining the composite key.

4.3. DELETE PROCEDURES

Delete procedures remove records based on primary key values, with minimal parameters.

EXAMPLE: DELETEMAINTENANCE.SQL

```

CREATE PROCEDURE DeleteMaintenance
    @Maintenance_Number INT
AS
BEGIN
    DELETE FROM Maintenance WHERE Maintenance_Number = @Maintenance_Number
END

```

VARIATIONS IN DELETE PROCEDURES

Delete procedures are very similar across entities, with minor differences:

- **DeleteCustomer.sql**: Deletes customer records with cascading effects on related data.
- **DeleteEmployee.sql**: Removes employee information and breaks associations with garages.
- **DeletePayment.sql**: Deletes payment records while maintaining rental records.
- **DeleteRental.sql**: Deletes rental records with potential cascading effects on payments.
- **DeleteReview.sql**: Removes customer feedback without affecting rental records.
- **DeleteVehicle.sql**: Deletes vehicle information with effects on maintenance and rental associations.

JUNCTION TABLE DELETE PROCEDURES

Junction table deletes require composite key handling:

- **DeleteVehicleNeedsMaintenance.sql**: Requires both Maintenance_Number and CarID parameters.
- **DeleteVehicleRental.sql**: Accepts both RentalID and CarID to identify the specific relationship to remove.

4.4. SELECT/GET PROCEDURES

Select procedures retrieve data and are often used for populating user interface components.

EXAMPLE: GETALLEMPLOYEES.SQL

```

CREATE PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT EmployeeID, Name, Role, Age, Salary, Enrollment_Date FROM Employee
END

```

The system includes fewer select procedures, as some entities rely on direct SQL queries in the application code rather than stored procedures.

4.5. PROCEDURE ORGANIZATION

The stored procedures are logically organized into folders by entity:

1. **CustomerProcedures:** Customer information management
2. **EmployeeProcedures:** Employee data operations
3. **MaintenanceProcedures:** Maintenance record management
4. **PaymentProcedures:** Payment transaction handling
5. **RentalProcedures:** Rental booking management
6. **ReviewProcedures:** Customer feedback processing
7. **GarageProcedures:** Manages garage-employee assignments and operational details like roles and vehicle counts.
8. **VehicleProcedures:** Vehicle information management
9. **VehicleMaintenanceProcedures:** Vehicle-maintenance relationships
10. **VehicleRentalProcedures:** Vehicle-rental relationships with pricing

5. STORED PROCEDURE TESTING

To ensure the reliability of the database operations, the system includes a comprehensive testing script (**TestProcedures.sql**) that validates the functionality of all stored procedures. This testing approach demonstrates how to properly call each procedure with appropriate parameters and verify its behavior.

5.1. TEST STRUCTURE AND ORGANIZATION

The test script is organized by entity, with each section testing the complete CRUD (Create, Read, Update, Delete) cycle for a specific entity. For each entity, the tests:

1. Insert a new record using the corresponding Insert procedure
2. Capture the newly generated ID
3. Update the record using the Update procedure
4. Delete the record using the Delete procedure

Each test section is clearly delineated with PRINT statements to identify the entity being tested, making the test results easy to understand.

5.2. CUSTOMER PROCEDURE TESTS

The script begins by testing the customer-related procedures:

```
-- Test Customer Procedures
PRINT 'Testing Customer Procedures...'
GO

-- Test InsertCustomer
DECLARE @NewCustomerID INT;
EXEC @NewCustomerID = InsertCustomer
    @FirstName = 'John',
    @LastName = 'Doe',
    @Email = 'john.doe@example.com',
    @Phone = '1234567890',
    @Address = '123 Main St',
    @LicenseNumber = 'DL123456',
    @LicenseExpiry = '2025-12-31';

PRINT 'New Customer ID: ' + CAST(@NewCustomerID AS VARCHAR(10));
```

```

-- Test UpdateCustomer
EXEC UpdateCustomer
    @CustomerID = @NewCustomerID,
    @FirstName = 'John',
    @LastName = 'Smith',
    @Email = 'john.smith@example.com',
    @Phone = '0987654321',
    @Address = '456 Oak St',
    @LicenseNumber = 'DL123456',
    @LicenseExpiry = '2025-12-31';

PRINT 'Customer Updated Successfully';

-- Test DeleteCustomer
EXEC DeleteCustomer @CustomerID = @NewCustomerID;
PRINT 'Customer Deleted Successfully';

```

This test demonstrates:

- How to declare variables to capture return values from procedures
- How to pass parameters to stored procedures
- The complete lifecycle management of a customer record

5.3. PAYMENT PROCEDURE TESTS

Similar patterns are used to test payment-related procedures:

```

-- Test Payment Procedures
PRINT 'Testing Payment Procedures...'
GO

-- Test InsertPayment
DECLARE @NewPaymentID INT;
EXEC @NewPaymentID = InsertPayment
    @RentalID = 1,
    @Amount = 100.00,
    @PaymentDate = '2024-05-01',
    @PaymentMethod = 'Credit Card',
    @Status = 'Completed';

PRINT 'New Payment ID: ' + CAST(@NewPaymentID AS VARCHAR(10));

-- Test UpdatePayment
EXEC UpdatePayment
    @PaymentID = @NewPaymentID,
    @RentalID = 1,
    @Amount = 150.00,
    @PaymentDate = '2024-05-01',
    @PaymentMethod = 'Credit Card',
    @Status = 'Completed';

```

```
PRINT 'Payment Updated Successfully';
```

```
-- Test DeletePayment
```

```
EXEC DeletePayment @PaymentID = @NewPaymentID;
```

```
PRINT 'Payment Deleted Successfully';
```

The payment tests validate financial transaction management, ensuring that monetary values are correctly processed.

5.4. MAINTENANCE PROCEDURE TESTS

For vehicle maintenance records:

```
-- Test Maintenance Procedures
```

```
PRINT 'Testing Maintenance Procedures...'
```

```
GO
```

```
-- Test InsertMaintenance
```

```
DECLARE @NewMaintenanceID INT;
```

```
EXEC @NewMaintenanceID = InsertMaintenance
```

```
    @VehicleID = 1,
```

```
    @MaintenanceDate = '2024-05-01',
```

```
    @Description = 'Regular oil change',
```

```
    @Cost = 50.00,
```

```
    @Status = 'Completed';
```

```
PRINT 'New Maintenance ID: ' + CAST(@NewMaintenanceID AS VARCHAR(10));
```

```
-- Test UpdateMaintenance
```

```
EXEC UpdateMaintenance
```

```
    @MaintenanceID = @NewMaintenanceID,
```

```
    @VehicleID = 1,
```

```
    @MaintenanceDate = '2024-05-01',
```

```
    @Description = 'Regular oil change and filter replacement',
```

```
    @Cost = 75.00,
```

```
    @Status = 'Completed';
```

```
PRINT 'Maintenance Updated Successfully';
```

```
-- Test DeleteMaintenance
```

```
EXEC DeleteMaintenance @MaintenanceID = @NewMaintenanceID;
```

```
PRINT 'Maintenance Deleted Successfully';
```

These tests validate the service record management functionality, including cost tracking.

5.5. REVIEW PROCEDURE TESTS

Customer feedback management is tested with:

```
-- Test Review Procedures
PRINT 'Testing Review Procedures...'
GO

-- Test InsertReview
DECLARE @NewReviewID INT;
EXEC @NewReviewID = InsertReview
    @RentalID = 1,
    @Rating = 5,
    @Comment = 'Great service!',
    @ReviewDate = '2024-05-01';

PRINT 'New Review ID: ' + CAST(@NewReviewID AS VARCHAR(10));

-- Test UpdateReview
EXEC UpdateReview
    @ReviewID = @NewReviewID,
    @RentalID = 1,
    @Rating = 4,
    @Comment = 'Very good service, but could be improved',
    @ReviewDate = '2024-05-01';

PRINT 'Review Updated Successfully';

-- Test DeleteReview
EXEC DeleteReview @ReviewID = @NewReviewID;
PRINT 'Review Deleted Successfully';
```

These tests ensure customer feedback can be properly recorded, modified, and removed if necessary.

5.6. RENTAL PROCEDURE TESTS

Finally, the core rental transaction functionality is tested:

```
-- Test Rental Procedures
PRINT 'Testing Rental Procedures...'
GO

-- Test InsertRental
DECLARE @NewRentalID INT;
EXEC @NewRentalID = InsertRental
    @StartDate = '2024-05-01',
    @EndDate = '2024-05-07',
    @Status = 'Active',
    @CustomerID = 1;

PRINT 'New Rental ID: ' + CAST(@NewRentalID AS VARCHAR(10));

-- Test UpdateRental
```



```

EXEC UpdateRental
    @RentalID = @NewRentalID,
    @StartDate = '2024-05-01',
    @EndDate = '2024-05-08',
    @Status = 'Extended',
    @CustomerID = 1;

PRINT 'Rental Updated Successfully';

-- Test DeleteRental
EXEC DeleteRental @RentalID = @NewRentalID;
PRINT 'Rental Deleted Successfully';

```

These tests validate the core business operations of creating and managing rental transactions.

5.7. TEST EXECUTION AND VERIFICATION

The test script is designed to be run as a batch, with each section executing in sequence. The PRINT statements provide visual feedback about the progress and success of each test operation. A successful test run will output a series of success messages and the IDs of newly created records.

This comprehensive testing approach ensures that all stored procedures function correctly before they are used in the production application. It also serves as documentation for how to properly call each procedure, including the required parameters and expected return values.

6. DASHBOARD - CENTRALIZED USER INTERFACE

The Vehicle Rental System application is organized around a central Dashboard that serves as the main entry point and navigation hub for all system functionality. The Dashboard implements a tab-based interface that brings together all the individual user controls into a cohesive application.

6.1. DASHBOARD IMPLEMENTATION

The Dashboard is implemented as a Windows Form that initializes and organizes all the user controls for the different entities in the system:

```

public partial class Dashboard : Form
{
    public Dashboard()
    {
        InitializeComponent();
        this.Load += DashboardForm_Load;
    }

    private void DashboardForm_Load(object sender, EventArgs e)
    {
        var customerControl = new CustomerControl();
        customerControl.Dock = DockStyle.Fill;
        tabCustomer.Controls.Clear();
        tabCustomer.Controls.Add(customerControl);

        var rentalControl = new RentalsControl();
        rentalControl.Dock = DockStyle.Fill;
        tabRental.Controls.Clear();
    }
}

```

```

        tabRental.Controls.Add(rentalControl);

        // ... Additional controls loaded
    }
}

```

Explanation:

- This Dashboard class is a central controller that coordinates and hosts all user controls within a tabbed interface
- The constructor is minimal, focusing on:
 1. Initializing the form components with `InitializeComponent()`, which sets up the visual elements defined in the form designer
 2. Wiring up the form's Load event to the DashboardForm_Load method using the `+=` event subscription syntax
- The `DashboardForm_Load` method is triggered when the form first loads and is responsible for:
 1. Creating instances of each specialized user control (e.g., CustomerControl, RentalsControl)
 2. Configuring the controls' appearance with `Dock = DockStyle.Fill` to ensure they expand to fill their container
 3. Clearing any existing controls from each tab with `tabControl.Controls.Clear()`
 4. Adding the newly created control to the appropriate tab with `tabControl.Controls.Add()`
- Each tab page (tabCustomer, tabRental, etc.) serves as a container for a specific functional control
- Using `DockStyle.Fill` ensures responsive layout that adapts when the form is resized
- The form uses lazy loading by instantiating controls only when the form loads rather than in the constructor
- This pattern minimizes memory usage on startup and improves initial load time
- The partial class keyword (`public partial class`) indicates that this class is split across multiple files, with the designer-generated code in a separate file

6.2. INTEGRATED CONTROLS

The Dashboard integrates all major functional areas of the application:

- **CustomerControl:** Manages customer information and profiles
- **RentalsControl:** Handles rental bookings and transactions
- **PaymentControl:** Processes and tracks payments
- **MaintenanceControl:** Records and schedules vehicle maintenance
- **GarageControl:** Manages garage information and employee assignments
- **VehicleControl:** Handles vehicle inventory and details
- **VehicleRentalControl:** Manages the relationship between vehicles and rentals
- **VehicleMaintenanceControl:** Tracks which vehicles need maintenance
- **EmployeeControl:** Manages employee information
- **ReviewsControl:** Handles customer feedback and reviews

6.3. CENTRALIZED ARCHITECTURE BENEFITS

This centralized architecture provides several key benefits:

1. **Unified User Experience:** Users access all functionality through a single interface with consistent navigation
2. **Modular Design:** Each functional area is encapsulated in its own control, enabling focused development and maintenance
3. **Separation of Concerns:** Business logic for each entity is isolated within its specific control
4. **Simplified Navigation:** The tab-based interface allows users to quickly switch between different system functions
5. **Consistent Visual Design:** The common container ensures a uniform appearance across all functional areas

The Dashboard serves as the glue that binds individual components together while maintaining their independence, demonstrating good architectural design principles and improving the overall user experience.
